

Powell-Style Model-Based Derivative-Free Optimization with Complexity Guarantees

A. Chaudhry[†] K. Scheinberg[‡] Scholar Sun[‡]

Abstract. We propose variants of model-based trust region derivative free algorithms that are closest to methods initially proposed and implemented by Powell in [22, 21]. These methods rely on low degree polynomial interpolation and carefully maintain geometry of the interpolation sets. We are able to derive complexity bounds for these methods that make them theoretically competitive to other derivative free methods. Applying these methods in randomly generated subspaces recovers what we believe to be nearly tight complexity. This paper builds on recent results in [11] where complexity of a much simplified version of Powell’s methods was derived. Here we extend the analysis to fully incorporate Powell’s geometry handling approach, and conduct extensive numerical comparison of the model-based trust region methods connecting practical and theoretical performance. We also extend the analysis of subspace model-based trust region methods initially developed in [11] to the case of noisy function evaluations.

MSC Classification: 90C30, 90C56.

1 Introduction. In this paper, we focus on the complexity of model-based derivative free algorithms that aim to solve unconstrained nonlinear optimization problems of the form

$$(1.1) \quad \min_{x \in \mathbb{R}^n} \phi(x),$$

where $\phi : \mathbb{R}^n \rightarrow \mathbb{R}$ is a smooth objective function that may not be convex. The key premise of these methods is to economize on function values, possibly at the expense of additional linear algebra, since in most applications the function evaluations cost dominates all else. The complexity will be measured in terms of the number of function evaluations needed to achieve an ϵ -stationary point, that is a point x for which $\|\nabla\phi(x)\| \leq \epsilon$.

The main goal of this paper is to narrow the gap between theory and practice in model based derivative free optimization (DFO). Derivative free optimization, also known under names *zeroth-order* and *gradient-free* optimization, is an area of optimization concerned with developing optimization methods based purely on function value computation without applying any direct differentiation. The area has experienced significant growth both in theory and in a variety of applications in the past several decades. There is a rich literature of DFO starting from around mid 90s which is rapidly growing with new interest spurred by new applications in engineering, machine learning and artificial intelligence. Aside from an increasing number of papers, there are two books [13] and [1], and a survey on the topic [19]. Examples of applications can be found in [1] and [25].

Model-based DFO methods approach the optimization problem by constructing and maintaining (usually local) models of the objective function from function value samples. A particular class of such methods, model-based trust region methods pioneered in the 90s by M.J.D. Powell [22, 21], proved to be very effective in practice for many applications [20]. These methods use polynomial interpolation models and maintain sample sets using carefully engineered techniques, supported by mathematical properties of Lagrange interpolation polynomials. The complex structure of the algorithms, especially as presented in Powell’s papers, made them difficult to implement, let alone to analyze rigorously. Some underlying theory of asymptotic convergence was developed in [13] and later complexity bounds were provided for those algorithms in [15], however, methods analyzed there are much simplified and include elements that significantly depart from Powell’s ideas. More specifically, Powell’s methods only use one or two function evaluations per iteration, attempting to select the sample points in an optimal way, to improve the objective value while also maintaining or improving geometry of sample points. In other words, they carefully balance exploration and exploitation. On the other hand, the algorithms which

[†]Department of Applied Mathematics, University of Colorado, Boulder

[‡]School of Industrial and Systems Engineering, Georgia Tech

have enjoyed complexity bounds so far require occasional complete model reconstruction, which requires far more function evaluations and essentially abandon Powell’s careful geometry correcting approach.

For the past several decades there was a general lack of understanding of how Powell’s methods work and most importantly if they enjoy favorable complexity bounds. Recently, in a considerable implementational effort Powell’s software has been reincarnated in modern platforms by Z. Zhang and his colleagues [24, 27]. Their software packages have seen considerable success and are now being widely used by practitioners. This raises the interest in providing solid theory for these methods. In [11] a *geometry correcting* method inspired by Powell’s algorithms was proposed and its complexity bounds derived. There were several important questions addressed in that paper.

- It was shown that a model-based TR method which performs only one or two function evaluations per iteration has good complexity guarantees in the case of general polynomials.
- In the case of linear polynomial interpolation the worst case complexity bound is $\mathcal{O}(n^2\epsilon^{-2})$ which matches those of other DFO methods, such as direct search or methods based on finite difference gradient approximation.
- It was shown that model-based trust region methods (Powell’s or other) can be applied within a random subspace algorithm which results in a further improvement in the worst case complexity bound to $\mathcal{O}(n\epsilon^{-2})$.

In this paper we extend the results of [11] in several important ways.

- The method in [11] still departs from Powell’s method in its geometry handling approach. We will explain the details of this when we describe the algorithm but the key difference is that the method in [11] discards some potentially useful sample points, which Powell’s methods do not. Including these points complicates the theory substantially, but we are able to provide such theory here.
- We derive the theory based on linear interpolation since this is sufficient for the first-order convergence analysis, however, with an easy modification we allow for quadratic models without any additional complexity cost.
- We extend the analysis to functions with deterministic noise, which is essential when addressing real DFO applications. Deterministic noise implies a lower bound on the best reachable optimality criterion and we derive such a lower bound.
- We extend the analysis of the random subspace DFO trust region method to the case of deterministic noise. As we will show, there is an additional complication which requires algorithmic modifications in this case (as opposed to the full-space case or noise-free subspace case).
- Finally, we provide a careful implementation of our ideas and demonstrate that our theoretically supported algorithm can match the performance of other trust region DFO methods, including those in [24, 27].

1.1 Preliminaries The following are the standard assumptions on $\phi(x)$ for our setting.

ASSUMPTION 1.1 (Lower bound on ϕ). *The function ϕ is bounded below by a scalar ϕ^* on $\mathbb{R}^n$.*

ASSUMPTION 1.2 (Lipschitz continuous gradient). *The function ϕ is continuously differentiable, and the gradient of ϕ is L -Lipschitz continuous on $\mathbb{R}^n$, i.e., $\|\nabla\phi(y) - \nabla\phi(x)\| \leq L\|y - x\|$ for all $(y, x) \in \mathbb{R}^n \times \mathbb{R}^n$.*

Throughout the paper, we assume that we do not have access to $\nabla\phi(x)$ or its approximation using any form of differentiation. Instead we have access to an inexact zeroth-order oracle $f(x) \approx \phi(x)$. Specifically, for all x we assume

$$|f(x) - \phi(x)| \leq \epsilon_f$$

for some $\epsilon_f > 0$.

Our main objective is to derive algorithms with the following type of guarantees. For a given $\epsilon > 0$, let $\mathcal{C}_\epsilon$ denote the total number of calls to oracle $f(x)$ performed by the given algorithm that guarantees reaching a point x_ϵ for which $\|\nabla\phi(x_\epsilon)\| \leq \epsilon$. Then we seek the following guarantees: $\forall \epsilon > \psi(\epsilon_f)$ $\mathcal{C}_\epsilon \leq \Psi(n, \epsilon)$, if the algorithm is deterministic, or $\mathbb{E}[\mathcal{C}_\epsilon] \leq \Psi(n, \epsilon)$, if the algorithm is stochastic. Here ψ is some function of the oracle noise that

gives the best achievable optimality criterion and Ψ is the complexity bound. We will make use of $\mathcal{O}()$ notation to suppress dependence on constants in upper bounds, $\Omega()$ notation to do the same in the case of lower bounds and $\Theta()$ to be used with an equality up to a constant factor. For all methods considered here as for all methods of similar type $\Psi() = \mathcal{O}(\epsilon^{-2})$. Thus the main focus of this work is in the dependence of $\Psi(n, \epsilon)$ on n and the dependence of $\psi()$ on n and ϵ_f . This reflects practical concerns with respect to derivative-free optimization, which in contrast to the usual derivative-based optimization has complexity dependence on n and is known not to scale well for large dimensional problems.

The paper is organized as follows: In Section 2 we present the analysis of the basic model-based trust region framework and present some key results based on what is known as fully-linear models that drive complexity analysis. In Section 3 we establish how these fully linear models arise in polynomial interpolation and quantify their properties in terms of Lagrange polynomials associated with the sample sets. In Section 4 we propose our main algorithm that uses Lagrange polynomials as a tool for model maintenance and analyze its complexity. Section 5 focuses on the subspace trust region method with inexact zeroth-order oracle. Finally, in Section 6 we describe implementational enhancements and present our computational comparisons.

2 Basic algorithm and elements of complexity analysis. We first present and analyze the trust region framework with the focus on its key elements.

As in most trust region algorithms, at every iteration $k \in \{0, 1, \dots\}$, we construct a quadratic model to approximate $\phi(x)$ near the iterate x_k

$$(2.1) \quad m_k(x_k + s) = \phi(x_k) + g_k(x_k)^T s + \frac{1}{2} s^T H_k(x_k) s.$$

The model is then minimized (approximately) over the trust region $B(x_k, \Delta_k)$ - a Euclidean ball around x_k of a radius Δ_k . In the paper we use the abbreviation $g_k := g(x_k) = \nabla m_k(x_k)$ and $H_k := H(x_k) = \nabla^2 m_k(x_k)$.¹

The following definition, also widely used in the literature [13], helps us identify the requirements on models m_k that are critical for convergence.

DEFINITION 2.1 (Fully-linear model). *Given a ball around point x of radius Δ , $B(x, \Delta)$, we say that model $m(x + s)$ is a κ_{ef}, κ_{eg} -fully linear model of $\phi(x + s)$ on $B(x, \Delta)$ if*

$$\|\nabla m(x) - \nabla \phi(x)\| \leq \kappa_{eg} \Delta$$

and

$$|m(x + s) - \phi(x + s)| \leq \kappa_{ef} \Delta^2$$

for all $\|s\| \leq \Delta$.

We now state the algorithmic framework where at each iteration updates are made based on progress and also on whether or not the model is known to be fully linear. We do not need to specify κ_{eg} and κ_{ef} constants in the algorithm but we assume they exist and are fixed throughout the iterations. In the framework below we do not even discuss how to verify if a model is κ_{ef}, κ_{eg} -fully-linear model, we simply assume such mechanism is given as an input. In the later section we will address this aspect in detail and incorporate it into the algorithm.

¹Note that the constant term $\phi(x_k)$ appears in the definition (2.1) but will not be needed in the algorithm, since only changes in the model value $m_k(x_k) - m_k(x_k + s)$ are of interest.

Algorithm 1: Trust region method based on fully-linear models

Inputs: A zeroth-order oracle $f(x) \approx \phi(x)$, a starting point x_0 , TR radius Δ_0 , and hyperparameters $\eta_1 \in (0, 1)$, $\eta_2 > 0$, and $\gamma \in (0, 1)$. Mechanism for establishing if a model is κ_{ef}, κ_{eg} -fully-linear for some fixed κ_{ef}, κ_{eg} .

for $k = 0, 1, 2, \dots$ **do**

- 1 Compute model m_k and a trial step $x_k + s_k$ where $s_k \approx \arg \min_s \{m_k(x_k + s) : s \in B(0, \Delta_k)\}$.
- 2 Compute the ratio ρ_k as

$$\rho_k = \frac{f(x_k) - f(x_k + s_k)}{m_k(x_k) - m_k(x_k + s_k)}.$$
- 3 Update the iterate and the TR radius as

$$(x_{k+1}, \Delta_{k+1}) \leftarrow \begin{cases} (x_k + s_k, \gamma^{-1} \Delta_k) & \text{if } \rho_k \geq \eta_1 \text{ and } \|g_k\| \geq \eta_2 \Delta_k, \\ (x_k, \Delta_k) & \text{else, if model is not FL in } B(x_k, \Delta_k). \\ (x_k, \gamma \Delta_k) & \text{otherwise.} \end{cases}$$
- 4 Perform some model improvement steps if appropriate.

We will make the following standard assumption on the models m_k and their minimization.

ASSUMPTION 2.2. 1. The trust region subproblem is solved sufficiently accurately in each iteration k so that $x_k + s_k$ provides at least a fraction of Cauchy decrease, i.e. for some constant $0 < \kappa_{fcd} < 1$

$$(2.2) \quad m_k(x_k) - m_k(x_k + s_k) \geq \frac{\kappa_{fcd}}{2} \|g_k\| \min \left\{ \frac{\|g_k\|}{\|H_k\|}, \Delta_k \right\}.$$

2. There exists a constant $\kappa_{bhm} > 0$ such that, for all x_k generated by Algorithm 1, the spectral norm of the Hessian of the model is bounded as

$$\|H_k\| \leq \kappa_{bhm}.$$

Condition (2.2) is commonly used in the literature and is satisfied by the Cauchy point with $\kappa_{fcd} = 1$. See [12, Section 6.3.2] for more details.

Algorithm 1 is a variant of a standard trust region method, well studied in the literature [12]. There are two key differences that appear in trust region methods specifically in the DFO context. The first one is the condition $\|g_k\| \geq \eta_2 \Delta_k$. This condition is not used in classical TR methods where $\nabla m_k(x_k) = g_k = \nabla \phi(x_k)$ and was first introduced in [3] for the case of a TR method based on random models. The reason for this condition is tied to the fact that the trust region radii have a dual function in this setting - controlling the step of the algorithm and also controlling the model accuracy. For the same reasons, most prior deterministic DFO literature relies on a much less practical and cumbersome "criticality step". The condition $\|g_k\| \geq \eta_2 \Delta_k$ ensures that the trust region radius and thus the gradient accuracy stays on track as the norm of the gradient reduces which removes the need for a separate criticality step. The second difference for the derivative-free model-based TR (versus the classical TR) method is the necessity to ensure that the model is fully-linear in the trust region when the trust region radius is reduced. One can simply assume that this is guaranteed, for example, by using finite difference gradient approximation to build the model at each iteration. This gives an easy and convenient way to analyze the algorithm, but this does not result in a practical method. In various prior works [22, 13, 11] efficient methods have been proposed to recognize whether a model is fully-linear and if not, to make an improving step. The purpose of this paper is to improve on these methods both in terms of theory and practice.

Step 3 of Algorithm 1 identifies three types of iterations: *successful* iteration, where the trial step is accepted and the trust region radius is increased, *unsuccessful* iteration, where the step is rejected and the trust region radius is decreased and what we will call *model improving* iteration where the step is rejected but the trust region radius is not decreased because the model is not fully-linear.

In this section we will derive the bound on the total number of successful and unsuccessful iterations and in the following section we will provide a mechanism for model improvement and will bound the number of the model improving iterations. For a given $\epsilon > 0$, let K_ϵ be the first iteration of Algorithms 1 for which $\|\nabla \phi(x_k)\| \leq \epsilon$. We

define the index sets

$$(2.3) \quad \begin{aligned} \mathcal{S}_\epsilon &:= \{k \in \{0, \dots, K_\epsilon - 1\} : \text{iteration } k \text{ is successful}\}. \\ \mathcal{M}_\epsilon &:= \{k \in \{0, \dots, K_\epsilon - 1\} : \text{iteration } k \text{ is model improving}\}. \\ \mathcal{U}_\epsilon &:= \{k \in \{0, \dots, K_\epsilon - 1\} : \text{iteration } k \text{ is unsuccessful}\}. \end{aligned}$$

We next present two lemmas that are key in the analysis of any trust region algorithm and specifically Algorithm 1. The first lemma establishes that once Δ_k is sufficiently small compared to the gradient norm, a successful step is ensured.

LEMMA 2.3 (small Δ_k implies successful step). *Under Assumption 2.2, if m_k is κ_{ef}, κ_{eg} -fully-linear and*

$$(2.4) \quad \sqrt{\frac{2\epsilon_f}{C_0}} \leq \Delta_k \leq C_1 \|\nabla\phi(x_k)\|, \text{ where } C_1 = \left(\max \left\{ \eta_2, \kappa_{bhm}, \frac{2\kappa_{ef} + C_0}{(1 - \eta_1)\kappa_{fcd}} \right\} + \kappa_{eg} \right)^{-1},$$

and C_0 is an arbitrary constant which we pick to equal $\max\{\eta_2, \kappa_{ef}\}$ for future convenience, then $\rho \geq \eta_1$, $\|g_k\| \geq \eta_2 \Delta_k$, thus iteration k is successful and $x_{k+1} = x_k + s_k$.

The proof is a simple modification of those in the stochastic trust region literature [6, 7]²

Proof. By the assumption that m_k is fully-linear, we have

$$\|\nabla\phi(x_k)\| \leq \|g_k\| + \kappa_{eg}\Delta_k.$$

From (2.4) we conclude that

$$\begin{aligned} (\max\{\kappa_{bhm}, \eta_2\} + \kappa_{eg})\Delta_k &\leq \|\nabla\phi(x_k)\| \leq \|g_k\| + \kappa_{eg}\Delta_k \\ \max\{\kappa_{bhm}, \eta_2\}\Delta_k &\leq \|g_k\|. \end{aligned}$$

This implies that the first condition of the successful step, namely $\|g_k\| \geq \eta_2 \Delta_k$, is satisfied. From (2.2) we have $m_k(x_k) - m_k(x_k + s_k) \geq \kappa_{fcd}\|g_k\|\Delta_k/2$. Thus, using the assumption that m_k is fully-linear again,

$$\begin{aligned} \rho_k &= \frac{m(x_k) - m(x_k + s_k) + (f(x_k) - m(x_k)) - (f(x_k + s_k) - m(x_k + s_k))}{m(x_k) - m(x_k + s_k)} \\ &= \frac{m(x_k) - m(x_k + s_k) + (\phi(x_k) - m(x_k)) - (\phi(x_k + s_k) - m(x_k + s_k))}{m(x_k) - m(x_k + s_k)} \\ &\quad + \frac{f(x_k) - \phi(x_k) + (f(x_k + s_k) - \phi(x_k + s_k))}{m(x_k) - m(x_k + s_k)} \\ &\geq 1 - \frac{\kappa_{ef}\Delta_k^2}{m(x_k) - m(x_k + s_k)} - \frac{2\epsilon_f}{m(x_k) - m(x_k + s_k)} \\ &\geq 1 - \frac{\kappa_{ef}\Delta_k^2}{\kappa_{fcd}\|g_k\|\Delta_k/2} - \frac{2\epsilon_f}{\kappa_{fcd}\|g_k\|\Delta_k/2} \geq 1 - \frac{(2\kappa_{ef} + 2\epsilon_f/\Delta_k^2)\Delta_k}{\kappa_{fcd}(\|\nabla\phi(x_k)\| - \kappa_{eg}\Delta_k)} \\ &\geq 1 - \frac{(2\kappa_{ef} + C_0)\Delta_k}{\kappa_{fcd}(\|\nabla\phi(x_k)\| - \kappa_{eg}\Delta_k)} \geq \eta_1, \end{aligned}$$

where the last inequality follows from (2.4) since $\|\nabla\phi(x_k)\| \geq (\frac{2\kappa_{ef}}{(1-\eta_1)\kappa_{fcd}} + \kappa_{eg})\Delta_k$. $\square$

LEMMA 2.4 (successful iteration implies function reduction). *Let Assumptions 1.2 and 2.2 hold. If the iteration k is successful, then*

$$(2.5) \quad \phi(x_k) - \phi(x_{k+1}) \geq C_2 \Delta_k^2 - 2\epsilon_f, \text{ where } C_2 = \frac{\eta_1 \eta_2 \kappa_{fcd}}{2} \min\left\{ \frac{\eta_2}{\kappa_{bhm}}, 1 \right\};$$

otherwise, we have $x_{k+1} = x_k$ and $\phi(x_k) - \phi(x_{k+1}) = 0$.

²This lemma was stated erroneously without proof in [11] with $4\epsilon_f$ instead of C_0 . The error is inconsequential, but we correct it here.

This proof can be found in [11]. Thus, under the additional assumption that $\Delta_k \geq \sqrt{\frac{2\epsilon_f}{\tau C_2}}$ for some $\tau \in (0, 1)$ (2.5) can be further stated as

$$(2.6) \quad \phi(x_k) - \phi(x_{k+1}) \geq (1 - \tau)C_2\Delta_k^2, \text{ where } C_2 = \frac{\eta_1\eta_2\kappa_{fcd}}{2} \min\left\{\frac{\eta_2}{\kappa_{bhm}}, 1\right\}.$$

Lemmas 2.3 and 2.4 give us a bound on the total number of successful and unsuccessful iterations, as long as it can be ensured that Δ_k remains not smaller than $\sqrt{\frac{2\epsilon_f}{\min\{\tau C_2, C_0\}}}$. Henceforth, for simplicity we use $\tau = \frac{1}{2}$, also since we chose $C_0 \geq \eta_2$ and $\eta_2 \geq C_2$ the bound on Δ_k reduces to $\sqrt{\frac{4\epsilon_f}{C_2}}$. Due to Lemma 2.3 and the update mechanism for Δ_k the bound is ensured as long as $\|\nabla\phi(x_k)\| \geq \epsilon$ for ϵ sufficiently large.

LEMMA 2.5 (Lower bound on Δ_k). *For any $\epsilon > \sqrt{\frac{4\epsilon_f}{\gamma^2 C_2 C_1^2}}$, assuming that $\Delta_0 > \gamma C_1 \epsilon$, for all $k \in \{0, \dots, K_\epsilon - 1\}$ $\Delta_k \geq \gamma C_1 \epsilon$.*

Proof. According to Lemma 2.3, any iteration $k \in \{0, \dots, K_\epsilon - 1\}$ must be successful when $\Delta_k \leq C_1 \epsilon$ and m_k is κ_{ef}, κ_{eg} -fully linear. Thus no iteration can be unsuccessful when $\Delta_k \leq C_1 \epsilon$. Thus, given $\Delta_0 > \gamma C_1 \epsilon$, and by the mechanism of Algorithm 1 we must have $\Delta_k \geq \gamma C_1 \epsilon$ for all $k \in \{0, \dots, K_\epsilon - 1\}$. $\square$

The following bound holds under the result of Lemma 2.5.

LEMMA 2.6 (Bound of successful iterations). *For any $\epsilon > \sqrt{\frac{4\epsilon_f}{\gamma^2 C_2 C_1^2}}$, assuming $\Delta_0 > \gamma C_1 \epsilon$, we have*

$$|\mathcal{S}_\epsilon| \leq \frac{2(\phi(x_0) - \phi^*)}{C_2(\gamma C_1 \epsilon)^2}$$

Proof. Using Lemma 2.4 with $\tau = \frac{1}{2}$ and from $\Delta_k \geq \gamma C_1 \epsilon$ for all $k \in \{0, \dots, K_\epsilon - 1\}$ we have

$$\phi(x_0) - \phi^* \geq \sum_{k=0}^{K_\epsilon-1} \phi(x_k) - \phi(x_{k+1}) \geq \frac{1}{2} \sum_{k \in \mathcal{S}_\epsilon} C_2 \Delta_k^2 > \frac{1}{2} |\mathcal{S}_\epsilon| C_2 (\gamma C_1 \epsilon)^2$$

which gives the result of the lemma. $\square$

LEMMA 2.7. *For any $\epsilon > \sqrt{\frac{4\epsilon_f}{\gamma^2 C_2 C_1^2}}$, assuming that the initial trust-region radius $\Delta_0 > \gamma C_1 \epsilon$,*

$$|\mathcal{U}_\epsilon| \leq |\mathcal{S}_\epsilon| + \lceil \log_\gamma \frac{C_1 \epsilon}{\Delta_0} \rceil.$$

Proof. We observe that

$$\Delta_{K_\epsilon} = \gamma^{-|\mathcal{S}_\epsilon|} \gamma^{|\mathcal{U}_\epsilon|} \Delta_0 \geq C_1 \epsilon,$$

The last inequality follows from the fact that $\Delta_{K_\epsilon-1} \geq \gamma C_1 \epsilon$ and the $K_\epsilon - 1$ -th iteration must be successful. Thus the number of unsuccessful iterations can be bounded using the number of successful ones rearranging the terms and taking the logarithm. $\square$

THEOREM 2.8. *Let Assumptions 1.2 and 2.2 hold. For any $\epsilon > \sqrt{\frac{4\epsilon_f}{\gamma^2 C_2 C_1^2}}$, assuming that the initial trust-region radius $\Delta_0 > \gamma C_1 \epsilon$, where C_1 is defined as $\left(\max\left\{\eta_2, \kappa_{bhm}, \frac{2\kappa_{ef} + \max\{\eta_2, \kappa_{ef}\}}{(1-\eta_1)\kappa_{fcd}}\right\} + \kappa_{eg} \right)^{-1}$, then we have the bound*

$$(2.7) \quad |\mathcal{S}_\epsilon| + |\mathcal{U}_\epsilon| \leq \frac{4(\phi(x_0) - \phi^*)}{C_2(\gamma C_1 \epsilon)^2} + \lceil \log_\gamma \frac{C_1 \epsilon}{\Delta_0} \rceil.$$

Constants C_1 and C_2 have a direct effect on the complexity and we will be using them (and their variations) throughout the paper. Let us pause here to understand their different components. Specifically, constants γ , η_1 and κ_{fcd} are usually chosen to be fixed in the algorithm and be close to 1 (say 0.9).

The key remaining constants are η_2 which is chosen in the algorithm and is used as specified, and κ_{ef} , κ_{eg} and κ_{bhm} which are all attributes of the constructed models and are not specified by the algorithm but are rather upper bounded by theory.

In what follows we will impose an upper bound on κ_{bhm} which will be dimension independent. Ideally κ_{bhm} should scale similarly to L , since the later is the bound on the norm of the true Hessian and the former is the bound on the model Hessian. Thus it is convenient to think of κ_{bhm} as $\sim \mathcal{O}(L)$, although it also can be zero if linear models are used but also can be large if allowed.

What remains is to derive concrete bounds on κ_{eg} , κ_{ef} with explicit dependence on dimension. These would depend on the manner in which the models are constructed.

The following standard lemma shows that a bound κ_{eg} (together with κ_{bhm}) implies a bound on κ_{ef} .

LEMMA 2.9 (Fully linear models). *Under Assumptions 1.2 and 2.2 if*

$$(2.8) \quad \|\nabla m(x) - \nabla \phi(x)\| \leq \kappa_{eg} \Delta$$

then $m_k(x_k + s)$ is a κ_{ef}, κ_{eg} -fully linear model of $\phi(x + s)$ on $B(x, \Delta)$ with $\kappa_{ef} = \kappa_{eg} + \frac{L + \kappa_{bhm}}{2}$.

Let us consider a concrete way of building fully-linear models and the complexity implications. From analysis of the finite difference gradient approximation error (see e.g. [4]), if one forms a gradient estimate via

$$(2.9) \quad g(x) = \sum_{i=1}^n \frac{f(x + \delta u_i) - f(x)}{\delta} u_i,$$

then one can derive a gradient approximation bound of

$$\|g(x) - \nabla \phi(x)\| \leq \frac{\sqrt{n}L\delta}{2} + \frac{2\sqrt{n}\epsilon_f}{\delta}.$$

Taking $\delta = \Delta_k$, at each iteration k , results in a κ_{ef}, κ_{eg} -fully linear model in $B(x_k, \Delta_k)$ with $\kappa_{eg} = \sqrt{n}L$ and $\kappa_{ef} = \frac{L + \kappa_{bhm}}{2} + \sqrt{n}L$, as long as $\Delta_k \geq 2\sqrt{\frac{\epsilon_f}{L}}$. To ensure this we add the lower bound $\gamma C_1 \epsilon \geq 2\sqrt{\frac{\epsilon_f}{L}}$ which translates to $\epsilon \geq \sqrt{\frac{4\epsilon_f}{\gamma^2 L C_1^2}}$.

The total bound on the number of function evaluations, i.e., oracle complexity, easily follows from Theorem 2.8 and from the fact that (2.9) gives a fully linear model at the cost of $n+1$ oracle calls. Thus there are no model improvement iterations and the final bound on the oracle complexity is derived via the bound on $(n+1)(|\mathcal{S}_\epsilon| + |\mathcal{U}_\epsilon|)$. As noted in [11], if η_2 is taken to be a constant independent of dimension, using this finite difference scheme, the total worse-case oracle complexity to achieve $\|\nabla \phi(x_k)\| \leq \epsilon$ for any $\epsilon > \sqrt{\frac{4\epsilon_f}{\gamma^2 \min\{C_2, L\} C_1^2}}$ is bounded by

$$\mathcal{C}_\epsilon \leq \mathcal{O}(n^2 \epsilon^{-2}).$$

In the following corollary, we show that this complexity is not optimal and can, in fact, be improved by taking η_2 to grow with the dimension of the problem by improving the bound on $|\mathcal{S}_\epsilon| + |\mathcal{U}_\epsilon|$ when κ_{ef} and κ_{eg} scale as $\mathcal{O}(\sqrt{n})$.

COROLLARY 2.10. *Under the same assumptions as Theorem 2.8, assuming that $g_k(x_k)$ is computed via (2.9) with $\delta = \Delta_k$ at each iteration, choosing $\eta_2 = \sqrt{n}$, and assuming $\kappa_{bhm} \leq \mathcal{O}(\sqrt{n})$, then for any $\epsilon > \sqrt{\frac{4\epsilon_f}{\gamma^2 \min\{C_2, L\} C_1^2}}$*

$$|\mathcal{S}_\epsilon| + |\mathcal{U}_\epsilon| \leq \mathcal{O}\left(\frac{\sqrt{n}}{\epsilon^2}\right)$$

and the total worst-case oracle complexity is bounded by

$$\mathcal{C}_\epsilon \leq \mathcal{O}\left(\frac{n^{3/2}}{\epsilon^2}\right)$$

Proof. By the error bound from [4], we have $\kappa_{ef}, \kappa_{eg} = O(\sqrt{n})$. Recall $C_1^{-1} = \max\left\{\eta_2, \kappa_{bhm}, \frac{2\kappa_{ef} + \max\{\eta_2, \kappa_{ef}\}}{(1-\eta_1)\kappa_{fed}}\right\} + \kappa_{eg}$ and $C_2 = \frac{\eta_1\eta_2\kappa_{fed}}{2} \min\left\{\frac{\eta_2}{\kappa_{bhm}}, 1\right\}$. Thus, under the assumptions of this corollary, we have $C_1^{-1} = \Theta(\sqrt{n})$ and $C_2 = \Theta(\sqrt{n})$. Thus, we can bound

$$(2.10) \quad \begin{aligned} |\mathcal{S}_\epsilon| + |\mathcal{U}_\epsilon| &\leq \frac{2(f(x_0) - f^*)}{C_2(\gamma C_1 \epsilon)^2} + \log_\gamma \frac{C_1 \epsilon}{\Delta_0} \\ &= \mathcal{O}\left(\frac{\sqrt{n}(\phi(x_0) - \phi^*)}{\epsilon^2} + \log\left(\frac{\sqrt{n}\Delta_0}{\epsilon}\right)\right) = \mathcal{O}\left(\frac{\sqrt{n}}{\epsilon^2}\right). \end{aligned}$$

The total oracle complexity follows as discussed before. $\square$

We note here that since we assume that L does not scale with n , while $C_1^{-1} = \Theta(\sqrt{n})$, then whether we choose η_2 to be constant or to equal $\sqrt{n}$ the lower bound on ϵ is $\Omega(\sqrt{n\epsilon})$.

In the next section we show how fully-linear models can be constructed via polynomial interpolation, using less rigid sample sets than used in (2.9) and yet we are able to derive a competitive bound on κ_{eg} (and thus κ_{ef}).

3 Lagrange polynomials and fully-linear models. Before introducing the method we wish to analyze in this paper we need to discuss an important tool utilized by these algorithms - Lagrange polynomials. The concepts and the definitions below can be found in [13].

Lagrange polynomials and associated concepts will be defined with respect to a space of polynomials $\mathcal{P}$ of dimension p . Typically $\mathcal{P}$ is either the set of linear or quadratic polynomials, but it also can be a set of quadratic polynomials with a pre-defined Hessian sparsity pattern.

DEFINITION 3.1. Lagrange Polynomials

Given a space of polynomials $\mathcal{P}$ of dimension p and a set of points $\mathcal{Y} = \{y_1, \dots, y_p\} \subset \mathbb{R}^n$, a set of p polynomials $\ell_j(s)$ in $\mathcal{P}$ for $j = 1, \dots, p$, is called a basis of Lagrange polynomials associated with $\mathcal{Y}$, if

$$\ell_j(y_i) = \delta_{ij} = \begin{cases} 1 & \text{if } i = j, \\ 0 & \text{if } i \neq j. \end{cases}$$

If the basis of Lagrange polynomials exists for the given $\mathcal{Y}$ then $\mathcal{Y}$ is said to be poised.

DEFINITION 3.2. Λ -poisedness Given a space of polynomials $\mathcal{P}$ of dimension p , $\Lambda > 0$, and a set $\mathcal{B} \subset \mathbb{R}^n$. A poised set $\mathcal{Y} = \{y_1, \dots, y_p\}$ is said to be Λ -poised in $\mathcal{B}$ if $\mathcal{Y} \subset \mathcal{B}$ and for the basis of Lagrange polynomials associated with $\mathcal{Y}$, it holds that

$$\Lambda \geq \max_{j=1, \dots, p} \max_{s \in \mathcal{B}} |\ell_j(s)|.$$

We now show how Λ -poisedness of the interpolation set can ensure that related interpolation model is fully linear and derive corresponding constants κ_{ef} and κ_{eg} . Throughout this section we apply Assumptions 1.2 and 2.2. By Lemma 2.9, all we need is to ensure (2.8). For this we specify a way to construct the model $m(x)$.

Let $\mathcal{Y}_k$ be a set of points and $\mathcal{P}$ be a space of polynomials. Assuming that $\mathcal{Y}_k = \{y_1, \dots, y_p\}$ is poised in $B(0, \Delta)$ we define g_k and H_k to satisfy $\|H\| \leq \kappa_{bhm}$ and

$$(3.1) \quad g_k^\top y + \frac{1}{2} y^\top H_k y = \phi(x_k + y) - \phi(x_k), \quad \forall y \in \mathcal{Y}_k.$$

The model $m_k(x)$ is then defined as

$$(3.2) \quad m_k(x_k + s) = \phi(x_k) + g_k^\top s + \frac{1}{2} s^\top H_k s.$$

We now show an important result that establishes a bound on κ_{eg} when $p = n$ and $\mathcal{Y}_k = \{y_1, \dots, y_n\}$ is Λ -poised for homogeneous linear interpolation. This result is an extension of a similar result in [11] which allows m_k to include a quadratic term.

THEOREM 3.3. Let $\mathcal{Y} = \{y_1, \dots, y_n\}$ be such that $\mathcal{Y}$ is Λ -poised in $B(0, \Delta)$. Let g, H satisfy $\|H\| \leq \kappa_{bhm}$ and $g^\top y + \frac{1}{2} y^\top H y = \phi(x + y) - \phi(x)$, for each $y \in \mathcal{Y}$. Then

$$\|\nabla \phi(x) - g\| \leq \frac{1}{2} (L + \kappa_{bhm}) \Delta \sqrt{n} \sqrt{n(\Lambda^2 - 1) + 2}.$$

In particular, if $\Lambda = 1 + O(1/n)$, then we will have $\|\nabla\phi(x) - g\| = O(\sqrt{n})\Delta$.

Proof. Let $\bar{\phi}(s) = \phi(x + s) - \phi(x)$, then $\bar{\phi}(0) = 0$ and $\nabla\phi(x) = \nabla\bar{\phi}(0)$. Let Y be the matrix with i th column equal to y_i , let D be the diagonal matrix such that $D_{ii} = \|y_i\|$, and let $\bar{\phi}(Y)$ be the vector with i th entry equal to $\bar{\phi}(y_i)$. The interpolation condition is

$$g^T y_i = \phi(x + y_i) - \phi(x) - \frac{1}{2} y_i^T H y_i, \quad i = 1, \dots, n$$

thus we have $g = Y^{-T} \bar{\phi}(Y) - Y^{-T} h(Y)$, where $h(Y)$ is a vector with components $\frac{1}{2} y_i^T H y_i$. Then we have $\|\nabla\phi(x) - g\| \leq \|\nabla\phi(x) - Y^{-T} \bar{\phi}(Y)\| + \|Y^{-T} h(Y)\|$. By the proof of Theorem 4.3 in [11], we have $\|Y^{-T} D\| \leq \sqrt{n(\Lambda^2 - 1) + 2}$ and

$$\|\nabla\phi(x) - Y^{-T} \bar{\phi}(Y)\| \leq \frac{1}{2} \sqrt{n} L \Delta \sqrt{n(\Lambda^2 - 1) + 2}$$

and we need only bound $\|Y^{-T} h(Y)\|$. We have $\|Y^{-T} h(Y)\| \leq \|Y^{-T} D\| \|D^{-1} h(Y)\|$. Observe that the i th element of $D^{-1} h(Y)$ is $\frac{1}{2} y_i^T H y_i / \|y_i\|$ and is therefore bounded by $\frac{1}{2} \kappa_{bhm} \Delta$. Now we have $\|D^{-1} h(Y)\| \leq \sqrt{n} \|D^{-1} h(Y)\|_\infty \leq \frac{1}{2} \sqrt{n} \kappa_{bhm} \Delta$. Thus $\|Y^{-T} h(Y)\| \leq \frac{1}{2} \sqrt{n} \kappa_{bhm} \Delta \sqrt{n(\Lambda^2 - 1) + 2}$, and in total

$$\|\nabla\phi(x) - g\| \leq \frac{1}{2} (L + \kappa_{bhm}) \Delta \sqrt{n} \sqrt{n(\Lambda^2 - 1) + 2}. \quad \square$$

This theorem allows us to conclude that our model is fully linear when our interpolation set is Λ -poised.

COROLLARY 3.4. *If m_k is defined as in (3.2) and if $\mathcal{Y}_k$ is Λ -poised, then m_k is κ_{ef}, κ_{eg} -fully-linear with*

$$\begin{aligned} \kappa_{eg} &= \frac{1}{2} (L + \kappa_{bhm}) \sqrt{n} \sqrt{n(\Lambda^2 - 1) + 2} \\ \kappa_{ef} &= \kappa_{eg} + \frac{L + \kappa_{bhm}}{2}. \end{aligned}$$

Proof. This follows from the definition of m_k in (3.2), Theorem 3.3 and Lemma 2.9. $\square$

We note that constructing the model to satisfy (3.1) is not possible unless $\phi(x + y) - \phi(x)$ can be computed exactly. In the case of inexact oracles we instead compute g and H from

$$(3.3) \quad g_k^\top y + \frac{1}{2} y^\top H_k y = f(x_k + y) - f(x_l) \quad \forall y \in \mathcal{Y}_k.$$

The previous theorem is modified as follows.

THEOREM 3.5. *Let $\mathcal{Y} = \{y_1, \dots, y_n\}$ be such that $\mathcal{Y}$ is Λ -poised in $B(0, \Delta)$. Let g, H be computed to satisfy (3.3) and $\|H\| \leq \kappa_{bhm}$. Then*

$$\|\nabla\phi(x) - g\| \leq \sqrt{n(\Lambda^2 - 1) + 2} \left(\frac{1}{2} (L + \kappa_{bhm}) \sqrt{n} \Delta + \sqrt{n} \frac{2\epsilon_f \Lambda}{\Delta} \right).$$

Proof. Define $\bar{\phi}$, D , $\bar{\phi}(Y)$, $h(Y)$ as in the previous proof. Diverging from that proof, the interpolation condition changes to

$$g^T y_i = \phi(x + y_i) - \phi(x) - \frac{1}{2} y_i^T H y_i + (f(x + y_i) - \phi(x + y_i)) - (f(x) - \phi(x)), \quad i = 1, \dots, n$$

thus we have $g = Y^{-T} \bar{\phi}(Y) - Y^{-T} h(Y) + Y^{-T} E$, where E is a vector with components $(f(x + y_i) - \phi(x + y_i)) - (f(x) - \phi(x))$. Then we can bound the error

$$\begin{aligned} \|\nabla\phi(x) - g\| &= \|Y^{-T} D(D^{-1} Y^T \nabla\bar{\phi}(0) - D^{-1} \bar{\phi}(Y) - D^{-1} E)\| + \|Y^{-T} h(Y)\| \\ &\leq \sqrt{n} \|Y^{-T} D\| \|D^{-1} Y^T \nabla\bar{\phi}(0) - D^{-1} \bar{\phi}(Y) - D^{-1} E\|_\infty + \|Y^{-T} h(Y)\| \\ &\leq \sqrt{n} \|Y^{-T} D\| (\|D^{-1} Y^T \nabla\bar{\phi}(0) - D^{-1} \bar{\phi}(Y)\|_\infty + \|D^{-1} E\|_\infty) + \|Y^{-T} h(Y)\| \\ &\leq \sqrt{n} \|Y^{-T} D\| (\|D^{-1} Y^T \nabla\bar{\phi}(0) - D^{-1} \bar{\phi}(Y)\|_\infty + \|D^{-1}\|_\infty \|E\|_\infty) + \|Y^{-T} h(Y)\|. \end{aligned}$$

We can bound $\|Y^{-T}D\|, \|D^{-1}Y^T\nabla\bar{\phi}(0) - D^{-1}\bar{\phi}(Y)\|_\infty, \|Y^{-T}h(Y)\|$ identically as in the previous proof. To bound $\|E\|_\infty$ observe that the condition that $|f(x+y) - \phi(x+y)| \leq \epsilon_f$ for $y \in \mathcal{Y} \cup \{0\}$ implies $\|E\|_\infty \leq 2\epsilon_f$. To bound $\|D^{-1}\|_\infty$ recall that $D_{ii} = \|y_i\|$. By the properties of Lagrange polynomials we have $\ell_i(y_i) = 1$. By linearity, we have $\ell_i(\Delta \frac{y_i}{\|y_i\|}) = \frac{\Delta}{\|y_i\|}$. By the poisedness condition we have $|\ell_i(\Delta \frac{y_i}{\|y_i\|})| \leq \Lambda$. Combining these bounds we have $\frac{\Delta}{\|y_i\|} \leq \Lambda$ and thus $\frac{1}{\|y_i\|} \leq \frac{\Lambda}{\Delta}$. Thus $\|D^{-1}\|_\infty \leq \frac{\Lambda}{\Delta}$. The result follows. $\square$

This theorem allows us to conclude that our model is fully linear when our interpolation set is Λ -poised and when Δ is sufficiently large.

COROLLARY 3.6. *If m_k is defined as in (3.2), if $\mathcal{Y}_k$ is Λ -poised, and if $\Delta_k \geq \sqrt{\frac{4\epsilon_f\Lambda}{L+\kappa_{bhm}}}$, then m_k is κ_{ef}, κ_{eg} -fully-linear with*

$$\begin{aligned}\kappa_{eg} &= (L + \kappa_{bhm}) \sqrt{n} \sqrt{n(\Lambda^2 - 1) + 2} \\ \kappa_{ef} &= \kappa_{eg} + \frac{L + \kappa_{bhm}}{2}.\end{aligned}$$

Proof. This follows from the definition of m_k in (3.2), Theorem 3.5 and Lemma 2.9, since the bound on Δ_k implies that

$$\sqrt{n(\Lambda^2 - 1) + 2} \left(\frac{1}{2} (L + \kappa_{bhm}) \sqrt{n} \Delta_k + \sqrt{n} \frac{2\epsilon_f\Lambda}{\Delta_k} \right) \leq 2 \frac{1}{2} (L + \kappa_{bhm}) \Delta_k \sqrt{n} \sqrt{n(\Lambda^2 - 1) + 2}.$$

In the next section we propose a concrete algorithm for constructing Λ -poised sample sets and deriving the bound on the number of iterations required to do so.

4 Model improving iterations based on Lagrange polynomials. In [11] a specific algorithm based on the framework of Algorithm 1 was proposed and its complexity analyzed. This algorithm maintains a set $\mathcal{Y}$ of n sample points and a set of linear Lagrange polynomials. At each model improving iteration the algorithm performs, what we call, a *geometry correcting step* by either replacing an interpolation point outside the trust region, if such a point exists, or replacing a point whose corresponding Lagrange polynomial violates the Λ -poisedness condition. If no such improvement is possible, then the set is Λ -poised and thus the model is fully-linear, by Theorems 3.3 and 3.5, hence the iteration is unsuccessful and the trust region radius is reduced. It is shown in [11] that the number of geometry correcting iterations between any two other iterations is at most $3n$. Each such iteration performs at most two function evaluations, while successful and unsuccessful iterations perform at most one. Thus using Theorem 2.8 one can derive the bound on $|\mathcal{M}_\epsilon|$ and consequently the bound on the total complexity of the algorithm.

The method in [11] fails to include several important practical elements. Firstly, the models that are being constructed by any successful model-based DFO method are quadratic and are usually based on quadratic interpolation. Secondly, since the function value is computed at the trial step, even if the step is not accepted as the new iterate, the step provides a new sample point which may improve the interpolation model. In fact it is likely to do so, since the reason the step is rejected, to begin with, is due to disagreement of the model and the function at the trial step. Thus adding this trial step to the new model provides new information. The improvement guaranteed by replacing some interpolation point by the trial step can be quantified by the value of the corresponding Lagrange polynomial at the trial step. A principled algorithm relying on this property, which is referred to as *self-correcting*, has been developed in [26] and shown to converge to a stationary point in the limit. This algorithm uses only such *self-correcting steps* for model improvement and does not perform geometry correction steps. It also includes the criticality step which is not practical but helps the analysis. No complexity bounds have been developed in [26]. Powell utilized both the self-correcting and geometry correcting steps in his algorithms [21, 23]. In what follows we present an algorithm that allows models to be constructed using quadratic interpolation and utilizes the self-correcting and geometry correcting steps. On the other hand it only maintains a set of linear Lagrange polynomials and ensures only a subset of interpolation points to be Λ -poised for linear interpolation. Combined with the results of the previous section this allows the algorithm to ensure eventual construction of fully-linear models. After we present the algorithm we state and prove the bound on the number of consecutive model improving iterations.

Algorithm 2 is the description of our proposed method that on the one hand tries to include most of the practical elements of Powell's methods and on the other hand nearly matches the complexity of the simplified method in [11].

At each iteration the method maintains two sets of points - set $\mathcal{Y}$ of n points whose geometry is monitored and maintained by means of the associated linear Lagrange polynomials and another set $\mathcal{Z}$ of $p \leq (n-1)n/2$ points whose geometry is only monitored and maintained in terms of distance to the trust region center.

We construct the model by solving the following constrained least squares problem.

$$\begin{aligned}
(4.1) \quad & \min_{g, H} \sum_{z \in \mathcal{Z}_k} \left(f(x_k) + g^\top z + \frac{1}{2} z^\top H z - f(x_k + z) \right)^2 \\
(4.2) \quad & \text{subject to: } \|H\| \leq K \\
(4.3) \quad & g^\top y + \frac{1}{2} y^\top H y = f(x_k + y) - f(x_k), \quad \forall y \in \mathcal{Y}_k.
\end{aligned}$$

Any solution to this problem will satisfy $\|H\| \leq \kappa_{bhm}$ with $\kappa_{bhm} \leq K$. If a quadratic model exists whose Hessian satisfies $\|H\| \leq \kappa_{bhm}$ with $\kappa_{bhm} \leq K$ and that interpolates all points in $\mathcal{Y}$ and $\mathcal{Z}$, then such model will be an optimal solution to this problem.³ When $p < \frac{n(n-1)}{2}$ then the problem may have multiple optimal solutions. In the case when this happens we can select the solution with the smallest Frobenius norm (see [13]), as long as it satisfies $\|H\| \leq K$. Alternatively, following Powell's ideas from [21] we can select the solution for which H is the closest in Frobenius norm to the Hessian from the previous iteration. We will discuss both of these techniques in the computational section.

³We deliberately distinguish K and κ_{bhm} , since K is chosen by the algorithm as will be set to be large, while κ_{bhm} is the true bound that occurs during the algorithm and will depend on the problem.

Algorithm 2: Geometry-correcting algorithm

Inputs: A zeroth-order oracle $f(x) \approx \phi(x)$, $p = \frac{(n-1)n}{2}$, $\Delta_0, x_0, \gamma \in (0, 1)$ $\eta_1 > 0, \eta_2 > 0, \Lambda > 1, \Lambda_{sc} \geq 1$.

Initialization An initial set $\mathcal{Y}_0$ such that $|\mathcal{Y}_0| = n$, an initial set $\mathcal{Z}_0$ such that $|\mathcal{Z}_0| \leq p$ and the function values $f(x_0), f(x_0 + y_i), y_i \in \mathcal{Y}_0, f(x_0 + z_i), z_i \in \mathcal{Z}_0$. A set of Lagrange Polynomials $\{\ell_i(x), i = 1, \dots, n\}$ in $\mathcal{P}$ for the set $\mathcal{Y}_0$.

for $k = 0, 1, 2, \dots$ **do**

1 Build a quadratic model $m_k(x_k + s)$ as in (3.2) using $f(x_k)$ and $f(x_k + y_i), y_i \in \mathcal{Y}_k, f(x_k + z_i), z_i \in \mathcal{Z}_k$.

2 Compute a trial step s_k and ratio ρ_k as in Algorithm 1.

3 *Successful iteration:* $\rho_k \geq \eta_1$ and $\|g_k\| \geq \eta_2 \Delta_k$.

Set $x_{k+1} = x_k + s_k, \Delta_{k+1} = \gamma^{-1} \Delta_k$.

$$j_k^* = \arg \max_{j=1, \dots, n} \|y_j - s_k\|, \quad i_k^* = \arg \max_{i=1, \dots, p} \|z_i - s_k\|.$$

If $\|y_{j_k^*} - s_k\| > \|z_{i_k^*} - s_k\|^a$ and $|\ell_{j_k^*}(s_k)| > 0$ then update set $\mathcal{Y}_{k+1} = (\mathcal{Y}_k \setminus \{y_{j_k^*}\} \cup \{0\}) - s_k$.

Recompute Lagrange Polynomials for $\mathcal{Y}_{k+1}$. Otherwise update set $\mathcal{Z}_{k+1} = (\mathcal{Z}_k \setminus \{z_{i_k^*}\} \cup \{0\}) - s_k$ if $|\mathcal{Z}_k| = p$,

or $\mathcal{Z}_{k+1} = \mathcal{Z}_k \cup \{0\} - s_k$ if $|\mathcal{Z}_k| < p$.

4 *Model improving or unsuccessful iteration:* $\rho_k < \eta_1$ or $\|g_k\| < \eta_2 \Delta_k$. Set $x_{k+1} = x_k, I_{imp} = 0$ and perform all the applicable steps below.

(i) *Geometry correction by replacing a far point in $\mathcal{Y}$:* Let $j_k^* = \arg \max_{j=1, \dots, n} \|y_j\|$.

If $\|y_{j_k^*}\| > \Delta_k \Rightarrow s^* = y_{j_k^*}, I_{imp} = 1$. If $|\ell_{j_k^*}(s_k)| > 0$, then $s_k^* = s_k$, otherwise $s_k^* = \arg \max_{s \in B(0, \Delta_k)} |\ell_{j_k^*}(s)|$, compute $f(x_k + s_k^*)$. Update $\mathcal{Y}_{k+1} = \mathcal{Y}_k \setminus \{y_{j_k^*}\} \cup \{s_k^*\}$ and Lagrange polynomials.

(ii) *Self-correction by replacing a point in $\mathcal{Y}$ with a large Lagrange Polynomial value at the trial step:* If $I_{imp} = 0$,

$$j_k^* = \arg \max_{j=1, \dots, n} |\ell_j(s_k)|.$$

If $|\ell_{j_k^*}(s_k)| > \Lambda_{sc}$, then $s^* = y_{j_k^*}$, update $\mathcal{Y}_{k+\frac{1}{2}} = \mathcal{Y}_k \setminus \{y_{j_k^*}\} \cup \{s_k\}$. Update the set of Lagrange Polynomials for $\mathcal{Y}_{k+\frac{1}{2}}$. Otherwise $s^* = s_k, \mathcal{Y}_{k+\frac{1}{2}} = \mathcal{Y}_k$.

(iii) *Geometry correction of $\mathcal{Y}$ by replacing a "bad" point:* If $I_{imp} = 0$ compute

$$(j_k^*, s_k^*) = \arg \max_{j=1, \dots, p, s \in B(0, \Delta_k)} |\ell_j(s)|.$$

If $|\ell_{j_k^*}(s_k^*)| > \Lambda$, compute $f(x_k + s_k^*)$, $\mathcal{Y}_{k+1} = \mathcal{Y}_{k+\frac{1}{2}} \setminus \{y_{j_k^*}\} \cup \{s_k^*\}$. Update the set of Lagrange Polynomials for $\mathcal{Y}_{k+1}$ and set $I_{imp} = 1$. Otherwise $\mathcal{Y}_{k+1} = \mathcal{Y}_{k+\frac{1}{2}}$.

(iv) *Attempt to improve $\mathcal{Z}$ using available points.* For each defined $s \in \{s^*, s_k^*\}$ repeat:

- *Improvement to $\mathcal{Z}$ by adding a point:* If $|\mathcal{Z}_k| < p, \mathcal{Z}_{k+1} = \mathcal{Z}_k \cup \{s\}$.
- *Improvement to $\mathcal{Z}$ to by replacing a far point:* Else, let $i_k^* = \arg \max_{i=1, \dots, n} \|z_i\|$. If $\|z_{i_k^*}\| > \|s\| \Rightarrow \mathcal{Z}_{k+1} = \mathcal{Z}_k \setminus \{z_{i_k^*}\} \cup \{s\}$.
- *Keep $\mathcal{Z}$:* Otherwise $\mathcal{Z}_{k+1} = \mathcal{Z}_k$.

Model improving iteration: If $I_{imp} = 1, \Delta_{k+1} = \Delta_k$.

Unsuccessful iteration: If $I_{imp} = 0, \Delta_{k+1} = \gamma \Delta_k$.

^aIf $\mathcal{Z}_k \neq \emptyset$ check $\|y_{j_k^*} - s_k\| > 0$

We note that case (iii) of Step 4 of Algorithm 2 is equivalent to one iteration of the geometry correction

procedure described in [13].

In cases (ii) and (iii) of Step 4 point $y_{j_k^*}$ in the current $\mathcal{Y}$ set is replaced by a new point, let's call it $\tilde{s}$. The update for deriving the new Lagrange Polynomial set ℓ^+ can be carried out via the following formulae:

$$(4.4) \quad \ell_{j_k^*}^+(x) = \frac{\ell_{j_k^*}(x)}{\ell_{j_k^*}(\tilde{s})}$$

$$(4.5) \quad \ell_i^+(x) = \ell_i(x) - \ell_i(\tilde{s})\ell_{j_k^*}^+(x) \quad i \neq j_k^*.$$

Note that since we assume that $\mathcal{Y}_0$ is poised then so are all consequent $\mathcal{Y}_k$ sets by construction.

We now provide results that allow us to bound the number of model improving iterations $\mathcal{M}_\epsilon$.

THEOREM 4.1. *Let $\mathcal{P}$ be the set of linear polynomials (with dimension $p = n$). Then the number of oracle calls in any sequence of consecutive model improving iterations; i.e. such that $k \in \mathcal{M}_\epsilon$ is at most $4n \log n + 8n + 2n \lfloor \log \log(\Lambda) \rfloor$.*

We break the proof of this theorem into the following two lemmas, the first of which derives a bound on the number of consecutive model improving iterations required to obtain a desired Λ -poised set starting from a Λ_0 -poised set for an arbitrary Λ_0 . The second lemma shows that after the first $2n$ consecutive model improving iterations Λ_0 -poised set is obtained with a particular bound on Λ_0 .

LEMMA 4.2. *Let $\mathcal{P}$ be the set of linear polynomials (with dimension $p = n$). If $\mathcal{Y}_k$ is Λ_0 -poised in $B(0, \Delta)$, then there will be at most $\lceil n \log n + n \lfloor \log \log \Lambda_0 \rfloor + n \lfloor \log \log(\Lambda) \rfloor \rceil$ additional consecutive model improving iterations.*

Proof. Assume for simplicity and w.l.o.g that $\Delta = 1$. Having a Λ_0 -poised set implies by Hadamard's inequality that we have

$$|\det(Y^{-T})| \leq \prod_{i=1}^n \|(Y^{-T})_i\| \leq \Lambda_0^n.$$

since $\max_{x \in B(0,1)} |\ell_i(x)| = \|(Y^{-T})_i\|$. Thus $|\det(Y)| \geq \Lambda_0^{-n}$. We have by Cramer's rule that replacing y_i with s results in a matrix Y^+ which satisfies

$$|\det(Y^+)| = |\det(Y)| |\ell_i(s)|.$$

Combining this with the fact that $|\det(Y)| \leq 1$, we have that $\max_{i \in [n]} \max_{x \in B(0,1)} |\ell_i(x)| \leq |\det(Y)|^{-1}$. Thus $\mathcal{Y}$ is Λ -poised if $|\det(Y)| \geq \Lambda^{-1}$. All that remains is to show that $|\det(Y)|$ must increase quickly.

After "Geometry correction of $\mathcal{Y}$ by replacing a "bad" point" resulting in a matrix Y^+ , we have

$$|\det(Y^+)| = |\det(Y)| \left(\max_{i \in [n]} \max_{x \in B(0,1)} |\ell_i(x)| \right) = |\det(Y)| \max_{i \in [n]} \|(Y^{-T})_i\|.$$

Observe by the AM-GM inequality

$$\max_{i \in [n]} \|(Y^{-T})_i\|^2 \geq \frac{1}{n} \sum_i \|(Y^{-T})_i\|^2 = \frac{1}{n} \|Y^{-T}\|_F^2 = \frac{1}{n} \sum_i \sigma_i(Y^{-T})^2 \geq \sqrt[n]{\prod_i \sigma_i(Y^{-T})^2} = |\det(Y)|^{-\frac{2}{n}}.$$

Thus we have $\max_{i \in [n]} \max_{x \in B(0,1)} |\ell_i(x)| = \max_{i \in [n]} \|(Y^{-T})_i\| \geq |\det(Y)|^{-\frac{1}{n}}$ and hence

$$|\det(Y^+)| \geq |\det(Y)| |\det(Y)|^{-\frac{1}{n}} = |\det(Y)|^{(1-\frac{1}{n})}.$$

Taking the logarithm of both sides, we have

$$\log |\det(Y^+)| \geq \left(1 - \frac{1}{n}\right) \log |\det(Y)|.$$

Thus, $-\log |\det(Y)|$ shrinks exponentially.

If in a single iteration we first do "Geometry correction by replacing a point in $\mathcal{Y}$ with a Lagrange Polynomial value" to get Y^+ , and then do "Geometry correction of $\mathcal{Y}$ by replacing a "bad" point" to get Y^{++} we have

$$\log |\det(Y^{++})| \geq \left(1 - \frac{1}{n}\right) \log |\det(Y^+)| \geq \left(1 - \frac{1}{n}\right) \log (|\det(Y)| |\ell_i(s_k^*)|) \geq \left(1 - \frac{1}{n}\right) \log |\det(Y)|.$$

Thus, in either case, $-\log |\det(Y)|$ shrinks exponentially.

Recall that we have $|\det(Y)| \geq \Lambda_0^{-n}$ and thus $\log |\det(Y)| \geq -n \log(\Lambda_0)$. Also, we will have achieved Λ -poisedness if $|\det(Y)| \geq \Lambda^{-1}$ or equivalently $\log |\det(Y)| \geq -\log(\Lambda)$. Thus, the number of additional iterations is bounded by

$$\left\lceil \frac{\log \left(\frac{-n \log(\Lambda_0)}{-\log(\Lambda)} \right)}{\log \left(1 - \frac{1}{n} \right)} \right\rceil \leq \left\lceil n \log \left(\frac{n \log(\Lambda_0)}{\log(\Lambda)} \right) \right\rceil \leq \lceil n \log n + n |\log \log \Lambda_0| + n |\log \log(\Lambda)| \rceil.$$

□

LEMMA 4.3. *Let $\mathcal{P}$ be the set of linear polynomials (with dimension $p = n$). Then $\mathcal{Y}_k$ is 14^n -poised after at most $2n$ consecutive model improving iterations.*

Proof. Because the first step of a model improving iteration replaces any point in $\mathcal{Y}_k$ that is outside $B(0, \Delta_k)$ then after at most n such iterations (and n oracle calls), $\mathcal{Y}_k$ contains n points all of which have norm at most Δ_k . For simplicity, we assume $\Delta_k = 1$. We show that in at most n additional iterations, the set $\mathcal{Y}_k$ becomes 14^n -poised.

For a subspace S , we say that a set of points $\mathcal{Y}$ is Λ -poised in S if for the Lagrange polynomials ℓ_i , we have $\max_{x \in B(0,1) \cap S} |\ell_i(x)| \leq \Lambda$. Note that if $y_1 \in \mathcal{Y}_k$ is a unit vector, then $\mathcal{Y}$ is 1-poised in $S = \text{span}(\{y_1\})$. Below, we prove two claims regarding poisedness in a subspace.

First, we claim that if $\mathcal{Y}$ is Λ -poised in S , then after step (ii): “Self-correction by replacing a point in $\mathcal{Y}$ with a large Lagrange Polynomial value”, we will have that the new set of points is 2Λ -poised in S .

Let s_k be the point we are adding and let $j_k^* = \arg \max_{j=1, \dots, n} |\ell_j(s_k)|$. For the replacement to take place, we must have $|\ell_{j_k^*}(s_k)| \geq 1$. Let ℓ_i^+ denote the Lagrange polynomials after replacement. Then from (4.4) we have

$$\max_{x \in B(0,1) \cap S} |\ell_{j_k^*}^+(x)| = \max_{x \in B(0,1) \cap S} \frac{|\ell_{j_k^*}(x)|}{|\ell_{j_k^*}(s_k)|} \leq \frac{\Lambda}{1} \leq 2\Lambda.$$

For $i \neq j_k^*$, we have

$$\begin{aligned} \max_{x \in B(0,1) \cap S} |\ell_i^+(x)| &= \max_{x \in B(0,1) \cap S} \left| \ell_i(x) - \frac{\ell_i(s_k) \ell_{j_k^*}(x)}{\ell_{j_k^*}(s_k)} \right| \\ &\leq \max_{x \in B(0,1) \cap S} |\ell_i(x)| + \left| \frac{\ell_i(s_k)}{\ell_{j_k^*}(s_k)} \right| \left(\max_{x \in B(0,1) \cap S} |\ell_{j_k^*}(x)| \right) \leq 2\Lambda. \end{aligned}$$

The last inequality is because $|\ell_i(s_k)| \leq |\ell_{j_k^*}(s_k)|$. This completes the proof of this first claim.

Second, we claim that if $\mathcal{Y}$ is Λ -poised in a proper subspace S , then after (iii): “Geometry correction of $\mathcal{Y}$ by replacing a “bad” point”, we will have that the new set of points is 7Λ -poised in S^+ , where S^+ is a subspace of one dimension greater than S .

Let $(j_k^*, s_k^*) = \arg \max_{j=1, \dots, p, s \in B(0, \Delta_k)} |\ell_j(s)|$. We will replace $y_{j_k^*}$ with s_k^* ; let ℓ_i^+ denote the Lagrange polynomials after replacement and apply (4.4).

If $\mathcal{Y}$ is 2Λ -poised in the whole space, then we have

$$\max_{x \in B(0,1)} |\ell_{j_k^*}^+(x)| = \max_{x \in B(0,1)} \frac{|\ell_{j_k^*}(x)|}{|\ell_{j_k^*}(s_k^*)|} = 1 \leq 7\Lambda.$$

Furthermore, for $i \neq j_k^*$, we have

$$\begin{aligned} \max_{x \in B(0,1)} |\ell_i^+(x)| &= \max_{x \in B(0,1)} \left| \ell_i(x) - \frac{\ell_i(s_k^*) \ell_{j_k^*}(x)}{\ell_{j_k^*}(s_k^*)} \right| \\ &\leq \max_{x \in B(0,1)} |\ell_i(x)| + \left| \frac{\ell_i(s_k^*)}{\ell_{j_k^*}(s_k^*)} \right| \left(\max_{x \in B(0,1)} |\ell_{j_k^*}(x)| \right) \leq 4\Lambda \leq 7\Lambda. \end{aligned}$$

Thus, we have that the points are 7Λ poised in the whole space and we can take S^+ to be any subspace of dimension one greater than S . Thus, the claim is shown if $\mathcal{Y}$ is 2Λ -poised in the whole space. For the remainder of the proof of the claim we may assume $\mathcal{Y}$ is not 2Λ -poised in the whole space.

Let P_S and $P_S^\perp$ denote the projections onto S and its orthogonal complement, respectively. By linearity, we have

$$2\Lambda < \|\ell(s_k^*)\|_\infty = \|\ell(P_S^\perp s_k^*) + \ell(P_S s_k^*)\|_\infty \leq \|\ell(P_S^\perp s_k^*)\|_\infty + \|\ell(P_S s_k^*)\|_\infty \leq \|\ell(P_S^\perp s_k^*)\|_\infty + \Lambda.$$

Thus, $\|\ell(P_S^\perp s_k^*)\|_\infty > \Lambda$. By homogeneity and optimality we have,

$$\frac{\|\ell(P_S^\perp s_k^*)\|_\infty}{\|P_S^\perp s_k^*\|} = \left\| \ell \left(\frac{P_S^\perp s_k^*}{\|P_S^\perp s_k^*\|} \right) \right\|_\infty \leq \|\ell(s_k^*)\|_\infty \leq \|\ell(P_S^\perp s_k^*)\|_\infty + \Lambda.$$

Thus we have

$$\|P_S^\perp s_k^*\| \geq \frac{\|\ell(P_S^\perp s_k^*)\|_\infty}{\|\ell(P_S^\perp s_k^*)\|_\infty + \Lambda} \geq \frac{1}{2}$$

since $\|\ell(P_S^\perp s_k^*)\|_\infty > \Lambda$.

By an argument identical to that used in the proof of the first claim, we have $\max_{x \in B(0,1) \cap S} \|\ell^+(x)\|_\infty \leq 2\Lambda$.

Next we wish to bound $\left\| \ell^+ \left(\frac{P_S^\perp s_k^*}{\|P_S^\perp s_k^*\|} \right) \right\|_\infty$. Noting that $\ell^+(s_k^*) = e_{j_k^*}$, we have

$$\|\ell^+(P_S^\perp s_k^*)\|_\infty = \|\ell^+(s_k^*) - \ell^+(P_S s_k^*)\|_\infty \leq \|\ell^+(s_k^*)\|_\infty + \|\ell^+(P_S s_k^*)\|_\infty = 1 + 2\Lambda \leq 3\Lambda.$$

Then we have

$$\left\| \ell^+ \left(\frac{P_S^\perp s_k^*}{\|P_S^\perp s_k^*\|} \right) \right\|_\infty = \frac{\|\ell^+(P_S^\perp s_k^*)\|_\infty}{\|P_S^\perp s_k^*\|} \leq 6\Lambda.$$

Let us define $S^+ = S \oplus \text{span}(\{P_S^\perp s_k^*\})$. We can represent any vector x in $B(0,1) \cap S^+$, as $\alpha x' + \beta \frac{P_S^\perp s_k^*}{\|P_S^\perp s_k^*\|}$ where $x' \in B(0,1) \cap S$ and $\alpha^2 + \beta^2 \leq 1$. Then we have

$$\left\| \ell^+ \left(\alpha x' + \beta \frac{P_S^\perp s_k^*}{\|P_S^\perp s_k^*\|} \right) \right\|_\infty \leq \alpha \|\ell^+(x')\|_\infty + \beta \left\| \ell^+ \left(\frac{P_S^\perp s_k^*}{\|P_S^\perp s_k^*\|} \right) \right\|_\infty \leq |\alpha|2\Lambda + |\beta|6\Lambda \leq \left\| \begin{bmatrix} 2 \\ 6 \end{bmatrix} \right\| \Lambda \leq 7\Lambda.$$

This completes the proof of the second claim.

For $t \leq n$, we will show by induction that after at most t iterations after removing far points, $\mathcal{Y}_k$ is 14^t -poised in a subspace of dimension at least t .

For the base case, when “Geometry correction of $\mathcal{Y}_k$ by replacing a "bad" point” is performed for the first time, it is done by maximizing a linear Lagrange polynomial over a ball. Since the maximum of a linear function over the unit ball is always attained on the boundary, the new point added to $\mathcal{Y}_k$ is on the boundary, thus $\mathcal{Y}_k$ contains at least one unit vector. Recall that if $y_1 \in \mathcal{Y}_k$ is a unit vector, then $\mathcal{Y}$ is 1-poised in $S = \text{span}(\{y_1\})$. Thus, $\mathcal{Y}_k$ is 1-poised in a one-dimensional subspace, establishing the base case.

Now let us assume that after the t th iteration after removing far points, $\mathcal{Y}_k$ is 14^t -poised in a subspace, S , of dimension at least t , and let us show that after an additional iteration, $\mathcal{Y}_k$ is 14^{t+1} -poised in a subspace of dimension at least $t+1$. In the t th-iteration, the algorithm either performs (ii) “Self-correction by replacing a point in $\mathcal{Y}_k$ with a large Lagrange Polynomial value” and (iii) “Geometry correction of $\mathcal{Y}_k$ by replacing a "bad" point”, or just the latter. If the algorithm performs both corrections, then after the first correction we have that the points are $2 \cdot 14^t$ -poised in S , by the first claim. Then after the second correction, by the second claim, the points are $7 \cdot 2 \cdot 14^t = 14^{t+1}$ -poised in S^+ which is of dimension at least one greater than S . Alternatively, if the algorithm only performs the latter correction, then by the second claim, the points are $7 \cdot 14^t \leq 14^{t+1}$ -poised in S^+ which is of dimension at least one greater than S . Thus, in either case, the inductive argument is complete. Thus, after at most n iterations after removing far points, $\mathcal{Y}_k$ is 14^n -poised in a subspace of dimension at least n (i.e. the whole space). Thus, after at most $2n$ total iterations (including removal of far away points), $\mathcal{Y}_k$ is 14^n -poised. $\square$

Proof of Theorem 4.1. We can combine Lemmas 4.3 and 4.2 by letting $\Lambda_0 = 14^n$. Thus the total number of iterations can be bounded as follows:

$$\begin{aligned} & 2n + \lceil n \log n + n |\log \log 14^n| + n |\log \log(\Lambda)| \rceil \\ &= 2n + \lceil 2n \log n + n |\log \log 14| + n |\log \log(\Lambda)| \rceil \\ &\leq 2n \log n + 4n + n |\log \log(\Lambda)|. \end{aligned}$$

Since each iteration uses at most 2 oracle calls, the result follows. $\square$

With these results, we may now present a total complexity theorem for Algorithm 2. Recall that Theorem 2.8 provides a bound for $|\mathcal{S}_\epsilon| + |\mathcal{U}_\epsilon|$. As we have just demonstrated the number of consecutive model improving iterations cannot be larger than $2n \log n + 4n + n \log \log(\Lambda)$ until set $\mathcal{Y}_k$ is Λ -poised in $B(x_k, \Delta_k)$. Let $\Lambda = 1 + \frac{1}{n}$, then the number of consecutive iterations can be bounded by $\mathcal{O}(n \log n)$. By Corollary 3.6 we have, for any $\Delta_k \geq \sqrt{\frac{4\epsilon_f \Lambda}{L + \kappa_{bhm}}}$,

$$\begin{aligned}\kappa_{eg} &= (L + \kappa_{bhm}) \sqrt{n} \sqrt{n(\Lambda^2 - 1) + 2} = \Theta(\sqrt{n}) \\ \kappa_{ef} &= \kappa_{eg} + \frac{L + \kappa_{bhm}}{2} = \Theta(\sqrt{n}).\end{aligned}$$

Thus we have the following corollary.

COROLLARY 4.4. *Under the same assumptions as Theorem 2.8, for any $\epsilon > \sqrt{\frac{4\epsilon_f}{\gamma^2 \min\{C_2, L + \kappa_{bhm}\} C_1^2}}$ (i.e. $\epsilon \geq \Omega(\sqrt{n\epsilon_f})$) the total oracle complexity of Algorithm 2 is bounded as*

$$\mathcal{C}_\epsilon \leq \mathcal{O}\left(\frac{n^{3/2} \log n}{\epsilon^2}\right)$$

if $\eta_2 = \sqrt{n}$ and as

$$\mathcal{C}_\epsilon \leq \mathcal{O}\left(\frac{n^2 \log n}{\epsilon^2}\right)$$

if η_2 is constant.

5 Model based trust region methods in subspaces. We now consider a trust region method where a model $m(x)$ is built and optimized in a random low-dimensional subspace of $\mathbb{R}^n$. The idea of using random subspace embeddings within derivative-free methods has gained a lot of popularity in the literature lately. It was shown in [16] that applying direct search methods in a low-dimensional subspace reduces the oracle complexity from $\mathcal{O}(n^2 \epsilon^{-2})$ to $\mathcal{O}(n \epsilon^{-2})$ (with dependence on the subspace dimensions suppressed). A random subspace version of a model-based TR method was first studied in [8] with the use of Johnson-Lindenstrauss (JL) subspace embeddings which achieved complexity $\mathcal{O}(n^2 \epsilon^{-2})$. Later in [14] this approach was combined with a stochastic model-based trust region method. Recently it was shown in [11] that a subspace model-based trust region method achieves an improved $\mathcal{O}(n \epsilon^{-2})$ complexity when the model is based on a random projection rather than a JL embedding. The key difference lies in the scaling of the projected gradient that is being estimated. A recent note [9] confirms that by rescaling the JL transformation in [8] the rate $\mathcal{O}(n \epsilon^{-2})$ can be achieved.

The works [8, 11] do not consider noisy function values. In [14] the noise in the function values is stochastic and is assumed to be reducible to any desired accuracy, dictated by the trust region radius, which is allowed to shrink to an arbitrarily small value. Here we extend the analysis of a subspace trust region method from [11] to accommodate fixed (deterministic) noise in the function oracle. The fundamental difficulty of doing so is that the algorithmic framework with random subspaces does not by itself guarantee a lower bound on the trust region radius, which is necessary for the analysis of the noisy function oracles, as we have seen in the sections above. As a consequence we need to introduce two algorithmic modifications - a relaxed step acceptance criterion and an enforced lower bound on the trust region radius.

We begin by introducing the subspace embedding of our problem. Given a matrix $Q \in \mathbb{R}^{n \times q}$, with $q \leq n$ and orthonormal columns, $QQ^T \nabla \phi(x)$ is an orthogonal projection of $\nabla \phi(x)$ onto a subspace spanned by the columns of Q (we will call it a subspace induced by Q). We also define a reduction of $\phi(x)$ to the subspace, given by Q around x : $\hat{\phi}(v) = \phi(x + Qv)$, $v \in \mathbb{R}^q$, which implies $Q \nabla \hat{\phi}(0) = QQ^T \nabla \phi(x)$. Similarly we define $\hat{m}(v) = m(x + Qv)$, $v \in \mathbb{R}^q$, which implies $Q \nabla \hat{m}(0) = QQ^T \nabla m(x)$.

We now present a modified trust-region algorithm that constructs models and computes steps in the subspace. At each iteration $k \in \{0, 1, \dots\}$ the algorithm chooses $Q_k \in \mathbb{R}^{n \times q}$ with orthonormal columns. The model m_k is defined as

$$(5.1) \quad m_k(x_k + Q_k v) = \phi(x_k) + g_k^T Q_k v + \frac{1}{2} v^T Q_k^T H_k Q_k v.$$

For any vector v , $g_k^T Q_k v = g_k^T Q_k Q_k^T Q_k v$, thus without loss of generality, we will assume that $Q_k Q_k^T g_k = g_k$, in other words, g_k lies in the subspace induced by Q_k . We define the trust region in the subspace induced by Q_k as $B_{Q_k}(x_k, \Delta_k) = \{z : z = x_k + Q_k v, \|v\| \leq \Delta_k\}$.

We will also need the definition of a fully linear model with respect to the subspace. We use the following definition which is the same as in [11].

DEFINITION 5.1 (Fully-linear model in a subspace). *Given a matrix with orthonormal columns $Q \in \mathbb{R}^{n \times q}$, let $B_Q(x, \Delta) = \{z : z = x + Qv, \|v\| \leq \Delta\}$. Let*

$$(5.2) \quad m(x + Qv) = \phi(x) + g^T Qv + \frac{1}{2} v^T Q^T H Q v$$

and $\hat{m}(v) = m(x + Qv)$, $v \in \mathbb{R}^q$. We say that model $m(x + Qv)$ is a κ_{ef}, κ_{eg} -fully linear model of $\phi(x + Qv)$ on $B_Q(x, \Delta)$ if

$$(5.3) \quad \|\nabla \hat{m}(0) - \nabla \hat{\phi}(0)\| \leq \kappa_{eg} \Delta$$

and

$$(5.4) \quad |\hat{m}(v) - \hat{\phi}(v)| \leq \kappa_{ef} \Delta^2$$

for all $\|v\| \leq \Delta$.

Here too (5.3) implies (5.4) with a specific value of κ_{ef} .

LEMMA 5.2 (Lemma 6.6 from [11]). *Under Assumptions 1.2 and 2.2, if (5.3) holds then m is κ_{ef}, κ_{eg} -fully linear on $B_Q(x, \Delta)$ with*

$$\kappa_{ef} = \kappa_{eg} + \frac{L_Q + \kappa_{bhm}}{2}$$

where L_Q is the Lipschitz constant of $Q Q^T \nabla \phi(x)$.

Algorithm 3: Trust region method based on fully-linear models in subspace

Inputs: Inexact zeroth order oracle $|f(x) - \phi(x)| \leq \epsilon_f$, minimum radius $\Delta_{\min}$, initial x_0 , $\Delta_0 \geq \Delta_{\min}$, initial matrix $Q_0 \in \mathbb{R}^{n \times q}$ with orthonormal columns and $\eta_1 \in (0, 1)$, $\eta_2 > 0$, and $\gamma \in (0, 1)$.

for $k = 0, 1, 2, \dots$ **do**

- 1 For the current Q_k compute model m_k as in (5.1).
 - 2 Compute a trial step $x_k + s_k$ where $s_k = Q_k v_k$ with $v_k \approx \arg \min_v \{m_k(x_k + Q_k v) : \|v\| \leq \Delta_k\}$.
 - 3 Compute the ratio ρ_k as

$$\rho_k = \frac{f(x_k) - f(x_k + s_k) + 2\epsilon_f}{m_k(x_k) - m_k(x_k + s_k)}.$$
 - 4 Update the iterate and the TR radius as

$$(x_{k+1}, \Delta_{k+1}) \leftarrow \begin{cases} (x_k + s_k, \gamma^{-1} \Delta_k) & \text{if } \rho_k \geq \eta_1 \text{ and } \|g_k\| \geq \eta_2 \Delta_k, \\ (x_k, \Delta_k) & \text{else, if the model is not fully-linear in } B_{Q_k}(x_k, \Delta_k). \\ (x_k, \max\{\gamma \Delta_k, \Delta_{\min}\}) & \text{otherwise.} \end{cases}$$
 - 5 Update the subspace

$$Q_{k+1} \in \mathbb{R}^{n \times q} \leftarrow \begin{cases} \text{random} & \text{if } \rho_k \geq \eta_1 \text{ and } \|g_k\| \geq \eta_2 \Delta_k \text{ or if a model is fully-linear} \\ Q_k & \text{otherwise.} \end{cases}$$
 - 6 Perform some model improvement steps.
-

We will assume, as before, that Assumption 2.2 holds. The two key modifications of Algorithm 3 compared to Algorithm 1 are the additional term $2\epsilon_f$ in the definition of ρ and the imposed lower bound on the trust region radius Δ_{min} . Both of these modifications are needed because the noisy zeroth-order oracle makes it essential for Δ_k to remain sufficiently positive to ensure fully linear models. Such a lower bound on Δ_k is ensured in the deterministic framework of Algorithm 1 for sufficiently large ϵ . But this is not the case when random subspaces are used because unsuccessful iterations can occur even if Δ_k is small, due to the subspace not being chosen well. We note that since ϵ_f is an upper bound on the error in the zeroth order oracle, any upper estimate of it can be used in the algorithm. Of course, unnecessarily large values will have an adverse effect on the resulting best achievable accuracy ϵ . Similarly, we will see that the choice of ϵ_f dictates the best choice for Δ_{min} .

We now introduce a definition from [11] of a measure of how well the subspace induced by Q aligns with the current gradient.

DEFINITION 5.3 (Well aligned subspace). *The subspace spanned by columns of Q is κ_g -well aligned with $\nabla\phi(x)$ for a given x if*

$$(5.5) \quad \|QQ^T\nabla\phi(x) - \nabla\phi(x)\| \leq \kappa_g\|\nabla\phi(x)\|$$

for some $\kappa_g \in [0, 1)$.

While condition (5.5) involves the gradient $\|\nabla\phi(x)\|$ it ultimately reduces to the properties of the subspace. Essentially, it requires that the gradient is not too close to being orthogonal to the subspace induced by Q . Similar conditions and terminology have been used in [8, 11].

We also recall the following related lemma (recalling that $g_k = Q_k Q_k^T \nabla\phi(x_k)$).

LEMMA 5.4 (Lemma 6.3 from [11]). *On iteration k , Q_k is κ_g -well aligned with $\nabla\phi(x_k)$ if and only if*

$$(5.6) \quad \|Q_k Q_k^T \nabla\phi(x_k)\|^2 \geq (1 - \kappa_g^2) \|\nabla\phi(x_k)\|^2.$$

Also, if $m(x_k + s)$ is κ_{ef}, κ_{eg} -fully linear model of $\phi(x_k + s)$ on $B_{Q_k}(x_k, \Delta_k)$ then

$$(5.7) \quad \|g_k - Q_k Q_k^T \nabla\phi(x_k)\| \leq \kappa_{eg} \Delta_k$$

We can now show the following lemma which is analogous to Lemma 6.4 from [11].

LEMMA 5.5 (sufficient condition for a successful step). *Under Assumptions 1.2 and 2.2, if Q_k is κ_g -well aligned with $\nabla\phi(x_k)$, $m_k(x_k + s)$ is a κ_{ef}, κ_{eg} -fully linear model of $\phi(x_k + s)$ on $B_{Q_k}(x_k, \Delta_k)$ and if*

$$(5.8) \quad \Delta_k \leq \sqrt{1 - \kappa_g^2} \tilde{C}_1 \|\nabla\phi(x_k)\|$$

where

$$\tilde{C}_1 = (\max \left\{ \eta_2, \kappa_{bhm}, \frac{2\kappa_{ef}}{(1 - \eta_1)\kappa_{fcd}} \right\} + \kappa_{eg})^{-1}$$

then $\rho_k \geq \eta_1$, $\|g_k\| \geq \eta_2 \Delta_k$, and $x_{k+1} = x_k + s_k$, i.e. the iteration k is successful.

Proof. Due to Lemma 5.4, specifically (5.7) by triangle inequality,

$$\|Q_k Q_k^T \nabla\phi(x_k)\| \leq \|g_k\| + \kappa_{eg} \Delta_k$$

and also due to Lemma 5.4

$$(5.9) \quad \Delta_k \leq \sqrt{1 - \kappa_g^2} \tilde{C}_1 \|\nabla\phi(x_k)\| \leq \tilde{C}_1 \|Q_k Q_k^T \nabla\phi(x_k)\|$$

By (5.9) we have

$$(\max \{ \kappa_{bhm}, \eta_2, \frac{2\kappa_{ef}}{(1 - \eta_1)\kappa_{fcd}} \} + \kappa_{eg}) \Delta_k \leq (\|g_k\| + \kappa_{eg} \Delta_k)$$

which implies

$$\max \{ \kappa_{bhm}, \eta_2 \} \Delta_k \leq \|g_k\|.$$

This establishes that $\|g_k\| \geq \eta_2 \Delta_k$ and also $m_k(x_k) - m_k(x_k + s_k) \geq \kappa_{fcd} \|g_k\| \Delta_k / 2$ by Assumption 2.2. Then, using the fact that $|f(x) - \phi(x)| \leq \epsilon_f$ and the fully linear assumption on m_k in the subspace induced by Q_k and recalling that $s_k = Q_k v_k$ we have

$$\begin{aligned} \rho_k &= \frac{m_k(x_k) - m_k(x_k + s_k) + (f(x_k) - m_k(x_k)) - (f(x_k + s_k) - m_k(x_k + s_k)) + 2\epsilon_f}{m_k(x_k) - m_k(x_k + s_k)} \\ &\geq \frac{m_k(x_k) - m_k(x_k + s_k) + (\phi(x_k) - m_k(x_k)) - (\phi(x_k + s_k) - m_k(x_k + s_k))}{m_k(x_k) - m_k(x_k + s_k)} \\ &\geq 1 - \frac{\kappa_{ef} \Delta_k^2}{m_k(x_k) - m_k(x_k + s_k)} \geq 1 - \frac{\kappa_{ef} \Delta_k^2}{\kappa_{fcd} \|g_k\| \Delta_k / 2} \\ &\geq 1 - \frac{2\kappa_{ef} \Delta_k}{\kappa_{fcd} (\|Q_k Q_k^T \nabla \phi(x_k)\| - \kappa_{eg} \Delta_k)} \geq \eta_1, \end{aligned}$$

where the last step is true because $\|Q_k Q_k^T \nabla \phi(x_k)\| \geq (\frac{2\kappa_{ef}}{(1-\eta_1)\kappa_{fcd}} + \kappa_{eg}) \Delta_k$ follows from (5.9). $\square$

Now we show a lower bound on progress made in successful iterations similar to Lemma 4.3 of [7].

LEMMA 5.6 (Progress made in a successful iteration). *Under Assumptions 1.2 and 2.2, in Algorithm 3, if $\rho_k \geq \eta_1$ and $\|g_k\| \geq \eta_2 \Delta_k$, then*

$$\phi(x_k) - \phi(x_{k+1}) \geq C_2 \Delta_k^2 - 4\epsilon_f,$$

where $C_2 = \frac{1}{2} \eta_1 \eta_2 \kappa_{fcd} \min \left\{ \frac{\eta_2}{\kappa_{bhm}}, 1 \right\}$.

Proof. Since $\rho_k \geq \eta_1$, we have

$$\eta_1 \leq \frac{f(x_k) - f(x_k + s_k) + 2\epsilon_f}{m_k(x_k) - m_k(x_k + s_k)} \leq \frac{\phi(x_k) - \phi(x_k + s_k) + 4\epsilon_f}{m_k(x_k) - m_k(x_k + s_k)},$$

which we can rearrange as $\phi(x_k) - \phi(x_k + s_k) \geq \eta_1 (m_k(x_k) - m_k(x_k + s_k)) - 4\epsilon_f$. Since $\|g_k\| \geq \eta_2 \Delta_k$, then by (2.2), we have

$$\eta_1 (m_k(x_k) - m_k(x_k + s_k)) \geq \frac{\eta_1 \kappa_{fcd}}{2} \|g_k\| \min \left\{ \frac{\|g_k\|}{\kappa_{bhm}}, \Delta_k \right\} \geq C_2 \Delta_k^2. \quad \square$$

From these results one could bound the number of needed iterations if one had a guarantee of having well-aligned subspaces, however we wish to cover the random case where the subspace is well-aligned only with some probability. Accordingly, we study the random subspace case next.

5.1 Complexity analysis under random subspace selection. Observe that the matrix Q_k , and the trust region radius, Δ_k do not change on model improving iterations. We define Q_t to be the t th random matrix Q_k and similarly Δ_t and x_t are the trust region radius and iterate which correspond to Q_t . In other words, index t counts iterations that follow either a successful or an unsuccessful iteration. In what follows we will essentially derive a bound for $|\mathcal{S}_\epsilon| + |\mathcal{U}_\epsilon|$ by examining what happens to the objective function and trust region radius after these iterations (since we know that after model improving iterations neither the TR radius nor the function value change).

We now define some relevant stochastic processes:

$$\begin{aligned} I_t &= \mathbb{1}\{Q_t \text{ is } \kappa_g\text{-well aligned with } \nabla \phi(x_t)\}, \\ A_t &= \mathbb{1}\{Q_t \text{ leads to a successful iteration i.e., } \Delta_{t+1} = \gamma^{-1} \Delta_t\}, \\ B_t &= \mathbb{1}\{\Delta_t > \hat{C}_1 \|\nabla \phi(x_t)\|\}, \end{aligned}$$

where $\hat{C}_1 = \sqrt{1 - \kappa_g^2} \tilde{C}_1$ for $\tilde{C}_1$ as defined in Lemma 5.5.

We will say that matrix Q_t is "true" if $I_t = 1$. Let T_ϵ be the first t such that the iterate x_t produced by Algorithm 3 achieves $\|\nabla \phi(x_t)\| \leq \epsilon$. Let $\mathcal{F}_{t-1}$ denote the σ -algebra generated by the first t matrices, $\mathcal{F}_{t-1} = \sigma(Q_0, Q_1, \dots, Q_{t-1})$. We note that the random variables x_t and Δ_t are measurable with respect to $\mathcal{F}_{t-1}$. We define m_t , s_t , ρ_t to be the last model, proposed step, and ratio, respectively, which correspond to the matrix Q_t . The random variables m_t , s_t and ρ_t are measurable with respect to $\mathcal{F}_t$. The random variable $T_\epsilon = \min\{t : \|\nabla \phi(x_t)\| \leq \epsilon\}$ is a stopping time adapted to the filtration $\{\mathcal{F}_{t-1}\}$.

ASSUMPTION 5.7. *There exists a $\theta \in (\frac{1}{2}, 1]$ such that*

$$\mathbb{P}\{I_t = 1 | \mathcal{F}_{t-1}\} \geq \theta.$$

By Lemma 6.7 of [11], if we take Q_t such that the subspace it induces is uniformly distributed, we can take $\theta \geq \frac{243}{443} > 1/2$ and $\kappa_g = \sqrt{1 - \frac{q}{10n}}$.

Note that $\sigma(B_t) \subset \mathcal{F}_{t-1}$ and $\sigma(A_t) \subset \mathcal{F}_t$, that is the random variable B_t is fully determined by matrices $Q_0, \dots, Q_{t-1}$ produced by the algorithm, while A_t is fully determined by the matrices $Q_0, \dots, Q_t$. The stochastic process described here has essentially the same dynamics as the process analyzed in [10] and [11] enabling us to reuse the results. The only differences are the presence of lower bound $\Delta_{\min}$ and the possible increase of $\phi(x_k)$ on some successful iterations. The lower bound does not alter the main properties of the dynamics of Δ_k , since by fixing ϵ to be sufficiently large with respect to $\Delta_{\min}$ we ensure that $\Delta_{\min} < \hat{C}_1 \|\nabla \phi(x_t)\|$ for $t = 0, 1, \dots, T_\epsilon - 1$. Thus we retain the key property which follows from Lemma 5.5:

$$A_t \geq I_t(1 - B_t),$$

in other words, if matrix Q_t is true and the trust region radius is sufficiently small, then the iteration is successful.

The increase of objective function on certain iterations is due to the relaxed definition of ρ_k and the error in the zeroth order oracle. Such situations have been previously analyzed for line (step) search in [5, 18] and trust-region method in [7]. The analysis here is simpler but the key idea is that the increase is bounded by $4\epsilon_f$ and occurs on iterations whose number is not too large compared to the number of iterations where function decreases. By ensuring that the decrease is sufficiently large to compensate for the increase, the results are derived. Below we present the analysis.

To bound the total number of successful and unsuccessful iterations we first bound the number of matrices that lead to successful iterations with large Δ . For that let $\bar{B}_t = \mathbb{1}\{\Delta_t \geq \gamma \hat{C}_1 \epsilon\}$ (note that $B_t = 1 \Rightarrow \bar{B}_t = 1$). Then from the dynamics of Algorithm 3 we have the bound similar to [10] (also used in [11]).

LEMMA 5.8. *Suppose $\epsilon > \sqrt{\frac{8\epsilon_f}{C_2\gamma^2\hat{C}_1^2}}$ For any $l \in \{0, \dots, T_\epsilon - 1\}$ and for all realizations of Algorithm 3, we have*

$$\sum_{t=0}^l \bar{B}_t I_t A_t \leq \sum_{t=0}^l \bar{B}_t A_t \leq \frac{\phi(x_0) - \phi^* + 4\epsilon_f(\sum_{t=0}^l (1 - \bar{B}_t)A_t)}{\frac{1}{2}C_2(\gamma(\hat{C}_1\epsilon))^2},$$

Proof. Since $\epsilon > \sqrt{\frac{8\epsilon_f}{C_2\gamma^2\hat{C}_1^2}}$, $\Delta_t \geq \gamma \hat{C}_1 \epsilon$ implies that $\Delta_t \geq \sqrt{\frac{8\epsilon_f}{C_2}}$ which in turn implies that for large successful iterations (corresponding to $\bar{B}_t A_t$), $\phi(x_t) - \phi(x_{t+1}) \geq \frac{1}{2}C_2\Delta_t^2$ by Lemma 5.6. For small successful iterations (corresponding to $(1 - \bar{B}_t)A_t$), by the same lemma, we have $\phi(x_t) - \phi(x_{t+1}) \geq -4\epsilon_f$. The result follows. $\square$

A useful lemma that easily follows from the dynamics is as follows.

LEMMA 5.9. *Suppose $\Delta_0 \geq \hat{C}_1 \epsilon$ and $\Delta_{\min} \leq \gamma \hat{C}_1 \epsilon$. For any $l \in \{0, \dots, T_\epsilon - 1\}$ and for all realizations of Algorithm 3, we have*

$$\sum_{t=0}^l B_t(1 - A_t) \leq \sum_{t=0}^l \bar{B}_t A_t + \log_\gamma \left(\frac{\hat{C}_1 \epsilon}{\Delta_0} \right).$$

The following result is shown in [10] under Assumption 5.7,

$$\mathbb{E} \left(\sum_{t=0}^{T_\epsilon-1} \bar{B}_t(1 - I_t) \right) \leq \frac{1-\theta}{\theta} \mathbb{E} \left(\sum_{t=0}^{T_\epsilon-1} \bar{B}_t I_t \right),$$

from which the following lemma is derived.

LEMMA 5.10. *Let Assumption 5.7 hold. Under the condition that $\theta > 1/2$, $\Delta_0 \geq \hat{C}_1 \epsilon$, and $\Delta_{\min} \leq \gamma \hat{C}_1 \epsilon$, we have*

$$\mathbb{E} \left(\sum_{t=0}^{T_\epsilon-1} B_t \right) \leq \frac{1}{2\theta-1} \left(\sum_{t=0}^{T_\epsilon-1} \bar{B}_t A_t + \log_\gamma \left(\frac{\hat{C}_1 \epsilon}{\Delta_0} \right) \right).$$

Finally the following lemma is shown in [10] for the stochastic processes I_t, A_t and B_t since $A_t \geq I_t(1 - B_t)$ and by the dynamics of Δ_t .

LEMMA 5.11. *Let Assumption 5.7 hold.*

$$\mathbb{E} \left(\sum_{t=0}^{T_\epsilon-1} (1 - B_t) \right) \leq \frac{1}{2\theta} \mathbb{E} [T_\epsilon].$$

Putting these lemmas together we obtain the final expected complexity result.

THEOREM 5.12. *Let Assumption 1.2, Assumption 2.2 and Assumption 5.7 hold. Then for any $\epsilon > \sqrt{\frac{16\epsilon_f}{(2\theta-1)^2 C_2 \tilde{C}_1^2 \gamma^2}}$, assuming an initial trust-region radius $\Delta_0 \geq \hat{C}_1 \epsilon$, and $\Delta_{\min} \leq \gamma \hat{C}_1 \epsilon$, let T_ϵ be the random stopping time for the event $\{\|\nabla \phi(x_t)\| \leq \epsilon\}$. We have the bound*

$$\mathbb{E} [T_\epsilon] \leq \frac{4\theta}{(2\theta-1)^2} \left(\frac{\phi(x_0) - \phi^*}{\frac{1}{2} C_2 (\gamma \hat{C}_1 \epsilon)^2} + \log_\gamma \left(\frac{\hat{C}_1 \epsilon}{\Delta_0} \right) \right)$$

where $\hat{C}_1 = \sqrt{1 - \kappa_g^2} \tilde{C}_1$ for $\tilde{C}_1$ as in (5.8) and C_2 as in Lemma 5.6.

Proof. Observe that $\sum_{t=0}^{T_\epsilon-1} (1 - \bar{B}_t) A_t \leq \sum_{t=0}^{T_\epsilon-1} (1 - B_t)$. Thus, from Lemmas 5.8 and 5.10, we have

$$\mathbb{E} \left(\sum_{t=0}^{T_\epsilon-1} B_t \right) \leq \frac{1}{2\theta-1} \left(\frac{\phi(x_0) - \phi^*}{\frac{1}{2} C_2 (\gamma \hat{C}_1 \epsilon)^2} + \log_\gamma \left(\frac{\hat{C}_1 \epsilon}{\Delta_0} \right) \right) + C_3 \left(\sum_{t=0}^{T_\epsilon-1} (1 - B_t) \right),$$

where $C_3 = \frac{4\epsilon_f}{(2\theta-1)^{\frac{1}{2}} C_2 (\gamma \hat{C}_1 \epsilon)^2}$. It follows that

$$\mathbb{E} [T_\epsilon] \leq \frac{1}{2\theta-1} \left(\frac{\phi(x_0) - \phi^*}{\frac{1}{2} C_2 (\gamma \hat{C}_1 \epsilon)^2} + \log_\gamma \left(\frac{\hat{C}_1 \epsilon}{\Delta_0} \right) \right) + (1 + C_3) \mathbb{E} \left(\sum_{t=0}^{T_\epsilon-1} (1 - B_t) \right).$$

Combining with Lemma 5.11, we obtain

$$\mathbb{E} [T_\epsilon] \leq \frac{1}{2\theta-1} \left(\frac{\phi(x_0) - \phi^*}{\frac{1}{2} C_2 (\gamma \hat{C}_1 \epsilon)^2} + \log_\gamma \left(\frac{\hat{C}_1 \epsilon}{\Delta_0} \right) \right) + (1 + C_3) \frac{1}{2\theta} \mathbb{E} [T_\epsilon].$$

The lower bound on ϵ implies that $C_3 \leq \theta - \frac{1}{2}$ and so $\frac{1+C_3}{2\theta} \leq \frac{2\theta+1}{4\theta}$. Thus we have

$$\frac{2\theta-1}{4\theta} \mathbb{E} [T_\epsilon] \leq \frac{1}{2\theta-1} \left(\frac{\phi(x_0) - \phi^*}{\frac{1}{2} C_2 (\gamma \hat{C}_1 \epsilon)^2} + \log_\gamma \left(\frac{\hat{C}_1 \epsilon}{\Delta_0} \right) \right). \quad \square$$

In order to use this theorem for effective complexity bounds, we must specify how to form models in a subspace. In the following subsections we discuss the two different approaches we used in the full space case - finite differences and interpolation based Λ -poised sets. The key difference now is in the lower bound on Δ_k imposed by $\Delta_{\min}$ rather than occurring automatically.

5.2 Building models in a subspace One can form a gradient estimate via a subspace version of (2.9). This can take the following form given in [11]:

$$(5.10) \quad \hat{g}(0) = \sum_{i=1}^q \frac{f(x + \delta Q u_i) - f(x)}{\delta} u_i$$

where u_i is the i th column of an orthogonal $q \times q$ matrix. Let us define $g(x) = Q \hat{g}(0)$.

By similar analysis as in [4], we can derive the bound

$$\|\nabla \hat{m}(0) - \nabla \hat{\phi}(0)\| \leq \frac{\sqrt{q} L \delta}{2} + \frac{2\sqrt{q} \epsilon_f}{\delta}.$$

Choosing $\delta = \Delta_k$, we have that m_k is κ_{ef}, κ_{eg} -fully linear model for

$$\kappa_{eg} = \frac{\sqrt{q}L}{2} + \frac{2\sqrt{q}\epsilon_f}{\Delta_{\min}^2}, \quad \kappa_{ef} = \kappa_{eg} + \frac{LQ + \kappa_{bhm}}{2}.$$

In this case there are no model improving iterations. Thus all iterations are either successful or unsuccessful and each iteration requires either q or $q + 1$ function evaluations. With these specifics we can give a final complexity bound for Algorithm 3. For simplicity of the presentation we will give the final bounds in terms of the key components, such as $n, \epsilon, \epsilon_f, L$ and $\Delta_{\min}$.

THEOREM 5.13. *Let Assumption 1.2 and 2.2 hold. When randomizing, take Q_t such that the subspace it induces is uniformly distributed with $q \geq 3$. For all $k = 0, 1, \dots, K_\epsilon - 1$, define $m_k(x_k + s)$ with g_k as in (5.10). Let the parameters $\eta_1, \eta_2, \kappa_{bhm}, \gamma$ be constants and assume $L \geq 1$. Then for $\epsilon > \Omega(\sqrt{n}(L + \frac{\epsilon_f}{\Delta_{\min}^2})(\sqrt{\epsilon_f} + \Delta_{\min}))$, assuming an initial trust-region radius $\Delta_0 \geq \Omega(\sqrt{\epsilon_f} + \Delta_{\min})$, let K_ϵ be the random stopping time for the event $\{\|\nabla\phi(x_k)\| \leq \epsilon\}$. We have the bound*

$$\mathbb{E}[K_\epsilon] \leq \mathcal{O}\left(\left(\frac{n}{\epsilon^2}\right)\left(L + \frac{\epsilon_f}{\Delta_{\min}^2}\right)^2\right)$$

where the “big- $\mathcal{O}$ ” notation suppresses constant factors and an additive logarithmic term.

Proof. Since we use (5.10) in every iteration, we have that k and t are equivalent. From the definition of $\tilde{C}_1$ and the bound on κ_{ef} , we have $\tilde{C}_1^{-1} = \Theta(\kappa_{eg}) = \Theta(\sqrt{q}(L + \frac{\epsilon_f}{\Delta_{\min}^2}))$. By Lemma 6.7 of [11], we have $\sqrt{1 - \kappa_g^2} = \Theta(\sqrt{\frac{q}{n}})$. Thus we have $\hat{C}_1^{-1} = \Theta(\sqrt{n}(L + \frac{\epsilon_f}{\Delta_{\min}^2}))$. Then the condition that $\epsilon > \sqrt{\frac{16\epsilon_f}{(2\theta-1)^2 C_2 \tilde{C}_1^2 \gamma^2}}$ from Theorem 5.12 becomes that $\epsilon > \Omega(\sqrt{n}(L + \frac{\epsilon_f}{\Delta_{\min}^2})\sqrt{\epsilon_f})$. The condition that $\Delta_{\min} \leq \gamma \hat{C}_1 \epsilon$, becomes $\epsilon > \Omega(\sqrt{n}(L + \frac{\epsilon_f}{\Delta_{\min}^2})\Delta_{\min})$. Combining these two bounds results in the condition $\epsilon > \Omega(\sqrt{n}(L + \frac{\epsilon_f}{\Delta_{\min}^2})(\sqrt{\epsilon_f} + \Delta_{\min}))$. Finally the condition that $\Delta_0 \geq \hat{C}_1 \epsilon$ becomes $\Delta_0 \geq \mathcal{O}(\sqrt{\epsilon_f} + \Delta_{\min})$. The expected iteration bound then follows directly from Theorem 5.12. $\square$

Here we note that the lower bound on ϵ , $\Omega(\sqrt{n}(L + \frac{\epsilon_f}{\Delta_{\min}^2})(\sqrt{\epsilon_f} + \Delta_{\min}))$, can be approximately optimized by taking $\Delta_{\min} = \Theta(\sqrt{\epsilon_f})$. The lower bound then becomes $\epsilon \geq \Omega(\sqrt{n\epsilon_f})$ with a rate of

$$\mathbb{E}[K_\epsilon] \leq \mathcal{O}\left(\frac{n}{\epsilon^2}\right).$$

We also note that each iteration requires only $\mathcal{O}(q)$ function evaluations. Thus the total expected complexity rate is

$$\mathbb{E}[C_\epsilon] \leq \mathcal{O}\left(\frac{nq}{\epsilon^2}\right).$$

5.3 Geometry-correcting algorithm in subspaces. We now describe a geometry-correcting version of Algorithm 3. This algorithm performs model improving steps of Algorithm 2 until either successful step is achieved or a fully linear model in the subspace is formed. At that point it terminates the work in that subspace and regenerates a new subspace as well as restarts the models using the initial sample sets $\mathcal{Y}_0, \mathcal{Z}_0$. This initialization choice is somewhat arbitrary and can be replaced by different initial sets. Each time, however, this requires computation of new function values for all points in the “initial” sample set. Our computational results show that this is quite expensive, if we use $\mathcal{Y}_0$ and $\mathcal{Z}_0$ that contain q points in each. We can delay resampling the random subspace until several successful or unsuccessful steps have been encountered and extend the theory to such strategies. However our computational results so far do not support an advantage of this approach. We can also choose $\mathcal{Y}_0$ to contain only 1 point and $\mathcal{Z}_0$ to be empty by modifying model improvement step and Lagrange polynomial computation to allow for incomplete sets. This modification is simple from the theory point of view but whether it can be practically competitive is yet unclear. Thus we retain the simplest approach for our analysis.

For this algorithm we can show the following complexity rate.

COROLLARY 5.14. *Under the same assumptions as Theorem 5.13, letting $\Lambda = 1 + \frac{1}{q}$ and $\Delta_{\min} = \sqrt{\epsilon_f}$, and $|\mathcal{Z}_0| = q$, for any $\epsilon > \Omega(\sqrt{n\epsilon_f})$, the expected total oracle complexity of Algorithm 4 is bounded as*

$$\mathcal{O}\left(\frac{nq \log q}{\epsilon^2}\right).$$

Algorithm 4: Geometry-correcting algorithm in subspace

Inputs: Inexact zeroth order oracle $|f(x) - \phi(x)| \leq \epsilon_f$, minimum radius $\Delta_{\min}$, initial x_0 , $\Delta_0 \geq \Delta_{\min}$, initial matrix $Q_0 \in \mathbb{R}^{n \times q}$ with orthonormal columns and $\eta_1 \in (0, 1)$, $\eta_2 > 0$, $\gamma \in (0, 1)$, $\Lambda > 1$, $\Lambda_{sc} \geq 1$.

Initialization Initial sets $\mathcal{Y}_0, \mathcal{Z}_0 \subset \mathbb{R}^q$ and the function values $f(x_0)$, $f(x_0 + Q_0 y_i)$, $y_i \in \mathcal{Y}_0$, $f(x_0 + Q_0 z_i)$, $z_i \in \mathcal{Z}_0$. A set of Lagrange Polynomials $\{\ell_i(v), i = 1, \dots, q\}$ in $\mathcal{P}$ for the set $\mathcal{Y}_0$.

for $k = 0, 1, 2, \dots$ **do**

- 1 For the current Q_k , build a quadratic model $\hat{m}_k(v) = m_k(x_k + Q_k v)$ as in (3.2) using $f(x_k)$ and $f(x_k + Q_k y_i)$, $y_i \in \mathcal{Y}_k$, $f(x_k + Q_k z_i)$, $z_i \in \mathcal{Z}_k$.
- 2 Compute a trial step $x_k + s_k$ where $s_k = Q_k v_k$ with $v_k \approx \arg \min_v \{m_k(x_k + Q_k v) : \|v\| \leq \Delta_k\}$.
- 3 Compute the ratio ρ_k as

$$\rho_k = \frac{f(x_k) - f(x_k + s_k) + 2\epsilon_f}{m_k(x_k) - m_k(x_k + s_k)}.$$

- 4 Update the iterate and the TR radius as

$$(x_{k+1}, \Delta_{k+1}) \leftarrow \begin{cases} (x_k + s_k, \gamma^{-1} \Delta_k) & \text{if } \rho_k \geq \eta_1 \text{ and } \|g_k\| \geq \eta_2 \Delta_k, \\ (x_k, \Delta_k) & \text{else, if } \mathcal{Y}_k \text{ is not } \Lambda\text{-poised,} \\ (x_k, \max\{\gamma \Delta_k, \Delta_{\min}\}) & \text{otherwise.} \end{cases}$$

- 5 Update the subspace

$$Q_{k+1} \in \mathbb{R}^{n \times q} \leftarrow \begin{cases} \text{random} & \text{if } \rho \geq \eta_1 \text{ and } \|g_k\| \geq \eta_2 \Delta_k \text{ or if } \mathcal{Y}_k \text{ is } \Lambda\text{-poised} \\ Q_k & \text{otherwise.} \end{cases}$$

- 6 Update the interpolation sets

$$\mathcal{Y}_{k+1}, \mathcal{Z}_{k+1} \leftarrow \begin{cases} \mathcal{Y}_0, \mathcal{Z}_0 & \text{if } \rho \geq \eta_1 \text{ and } \|g_k\| \geq \eta_2 \Delta_k \text{ or if } \mathcal{Y}_k \text{ is } \Lambda\text{-poised} \\ \text{Update as in Step 4 of Algorithm 2} & \text{otherwise.} \end{cases}$$

Proof. By Theorem 3.5, since we chose $\Lambda = 1 + \frac{1}{q}$, we have the error bound

$$\|\nabla \hat{m}_k(0) - \nabla \hat{\phi}(0)\| \leq \mathcal{O}\left(\sqrt{q} \left(L \Delta_k + \frac{\epsilon_f}{\Delta_k}\right)\right).$$

Thus for iterations when $\mathcal{Y}_k$ is Λ -poised, we have that m_k is κ_{ef}, κ_{eg} -fully linear model for

$$\kappa_{eg}, \kappa_{ef} = \mathcal{O}\left(\sqrt{q} \left(L + \frac{\epsilon_f}{\Delta_{\min}^2}\right)\right).$$

From this, Theorem 5.12, and the arguments from the proof of Theorem 5.13, we can bound the number of non-geometry-correcting iterations (i.e. iterations where Q_k is resampled) by $\mathcal{O}(\frac{n}{\epsilon^2})$. For these iterations, we use only one oracle call to evaluate $f(x_k + s_k)$, however for the iteration which immediately follows a successful or an unsuccessful iteration, we use $\mathcal{O}(q)$ oracle calls to evaluate $f(x_{k+1} + Q_{k+1} y_i)$ and $f(x_{k+1} + Q_{k+1} z_i)$. For all other iterations, as in Algorithm 2, we use at most 2 oracle calls. Finally, by Theorem 4.1, the maximum number of consecutive geometry correcting iterations is $\mathcal{O}(q \log q)$. The result follows. $\square$

6 Numerical Implementations and Results In this section we propose an implementation of Algorithm 2, which incorporates all its elements such as the self-correcting and geometry-correcting steps but in addition includes several practical features. Some of these features are borrowed from Powell's algorithms and some are new. As we will discuss, all these additional features improve practical performance but make the analysis more cumbersome. However, ultimately the order of the worst-case complexity of the algorithm is preserved.

The practical implementation is given in Algorithm 5. It utilizes the two interpolation sets $\mathcal{Y}$ and $\mathcal{Z}$ to manage linear Lagrange polynomials while fitting quadratic models, as proposed in Algorithm 2. We compare our proposed algorithm to NEWUOA [21], which is arguably the most scalable of Powell's algorithms and which maintains geometry of the full interpolation set by the use of quadratic Lagrange polynomials, and with DFOTR [2] which is a surprisingly efficient method that does not maintain any Lagrange polynomials and only updates the sample set based on the distance of the points to the TR center. In that respect DFOTR also maintains two separate sets, in that it does not reduce the trust region on steps that are not successful and when there are fewer than $n + 1$ points in the appropriate vicinity of the trust region center.

We also test a variation of Algorithm 2, to which current theory does not extend and which uses the quadratic Lagrange polynomials combined with a NEWUOA-like self-correcting rule in place of the rule described in Algorithm 2. As this method seems to provide improvement in high-dimensional setting, it gives motivation for the theory from Sections 3 and 4 to be extended to quadratic Lagrange polynomials in future work. All solvers will use a novel, adaptive fitting scheme. We test these algorithms on a collection of unconstrained problems from the CUTEst test set [17] and investigate results in low dimension, high dimension, and in randomized subspaces.

The following Section 6.1 describes the design choices of our proposed algorithms and how it still satisfies the theory. In Section 6.2 we discuss the testing methodology and numerical results.

6.1 Practical Implementations Algorithm 5 (GC-YZ-LIN) is a practical implementation of the geometry correction framework which maintains linear Lagrange polynomials for the set $\mathcal{Y}$ while interpolating a quadratic model using $\mathcal{Y} \cup \mathcal{Z} \cup \{x_k\}$. This method can be seen as the middle ground between DFOTR and NEWUOA, where the former makes only minimal effort to ensure good sample set geometry while the latter uses a significant amount of effort.

We now describe the changes implemented in Algorithm 5 as opposed to Algorithm 2. Algorithm 5 makes use of a resolution floor and small step gate, both of which are ideas borrowed from Powell's methods.

The resolution floor σ_k is an adaptive lower bound on Δ_k which only gets decreased once no progress can be made for that resolution. That is, if $\Delta_k = \sigma_k$, the geometry is good, and the iteration is still unsuccessful we reset $\sigma_{k+1} = \theta \sigma_k$ for some $0 < \theta < 1$. We then allow Δ_k to shrink even on model improving iterations as long as Δ_k is larger than the floor σ_k . This is motivated by the observation that in higher dimensions it can be expensive to always ensure good geometry before shrinking, hence an alternative is to only ensure this good geometry at intervals throughout the trajectory of the algorithm.

A small step gate and small model gradient gate are used by checking the conditions $\|s_k\| \geq c_g \sigma_k$ and $\|g_k\| \geq \eta_2 \Delta_k$ respectively. The small step gate adds an additional condition under which an iteration is deemed successful, which we note improves the termination speed of the algorithm. In the small model gradient gate, before evaluating a proposed step, we check if the norm of the model gradient is small relative to the trust region radius. That is if we know the step is going to be rejected, we save a function evaluation by skipping the evaluation of the trial step and attempt a geometry correcting step and possibly shrink the trust region radius. The same principle applies for the small step gate.

With these two changes, the complexity analysis in Section 2 follows through with small modifications, provided that the following simple assumption holds.

ASSUMPTION 6.1. *On every iteration k , we have*

$$\|s_k\| \geq \kappa_{\text{step}} \min \left\{ \Delta_k, \frac{\|g_k\|}{\kappa_{bhm}} \right\}.$$

Note that if the trust region subproblem is solved exactly, the assumption is satisfied with $\kappa_{\text{step}} = 1$, while for the Cauchy step we have $\kappa_{\text{step}} = \frac{\kappa_{fcd}}{3}$. Then, for Lemma 2.3 to go through, we require that $c_g \leq \kappa_{\text{step}}$ and Assumption 6.1 to hold. It would then follow that

$$\kappa_{bhm} \Delta_k \leq \max\{\kappa_{bhm}, \eta_2\} \Delta_k \leq \|g_k\| \implies \min \left\{ \Delta_k, \frac{\|g_k\|}{\kappa_{bhm}} \right\} = \Delta_k,$$

hence $\|s_k\| \geq \kappa_{\text{step}} \Delta_k \geq c_g \sigma_k$, and a small Δ_k still implies a successful iteration, even with a small step gate. The rest of the complexity argument for the full-dimensional method follows very closely to the analysis in Section 2, by applying key results with σ_k instead of Δ_k and deriving a bound on the number of unsuccessful iterations until

Δ_k reaches the floor σ_k for each round of σ_k reductions. Hence the proposed method will obey the theory, at the cost of some logarithmic factors.

The model fitting procedure fits a quadratic model, which we call the “hedge” model, using the following somewhat elaborate sequence of steps. On each iteration we compute the minimum Frobenius norm (MFN) model and the minimum change Frobenius norm (MCFN) model. Both of these models are computed by solving

$$(6.1) \quad \min_{g, H} \|H_{prev} - H\|_F \quad \text{s.t.} \quad v^\top g + \frac{1}{2} v^\top H v = f(x_k + v) - f(x_k), \quad v \in \mathcal{Y} \cup \mathcal{Z}.$$

where the MFN model sets $H_{prev} = 0$ and the MCFN model sets H_{prev} to the previous iteration’s MCFN Hessian. For both models, an exponentially weighted moving average of a relative error score is maintained and is used to determine which model is to be used for the current iteration. Only one model is “active” at a time and if the error for the alternative model becomes lower than the active model, we switch which model is used to compute the trial step. This change comes from the observation that sometimes it may be beneficial to remember the current local curvature, while at other times it may be beneficial to fit a fresh model as previous model Hessians can become stale. The proposed scoring method is a way to adaptively decide which model is best suited for the current iteration. We remark that a similar idea was proposed in Powell’s original paper, where consecutive poor steps would trigger an MFN model to be fit instead of MCFN.

After this modeling procedure, to ensure a bounded model Hessian, we check if the fitted Hessian exceeds the bound, i.e. if $\|H\| > K$. If so, we fit g_k and H_k by solving the regression problem (4.1). We note that there are other ways to bound $\|H_k\|$, such as via clipping. Since several of the objective functions in our data set naturally have extreme curvature, to prevent the truncation of useful curvature information, we set K to be a very large number. As a result, the condition $\|H\| > K$ will rarely trigger, which will yield either one of the MFN or MCFN models throughout an overwhelming majority of the iterations. We note that setting K very large implies that for some problems κ_{bhm} becomes very large. However, it appears to happen only when L is similarly large, thus having large κ_{bhm} or reducing it has no bearing on the order of the theoretical complexity bound.

We also develop a version of Algorithm 5, which we call GC-YZ-V, where the self-correction step computes and updates quadratic Lagrange polynomials over the set $\mathcal{Y} \cup \mathcal{Z}$, similar to NEWUOA, while still maintaining the $\mathcal{Y}$ - $\mathcal{Z}$ separation. Specifically, for the self-correcting step, we compute the score of a point in the sample set to be:

$$\text{score}(y_i) = \max \left(1, \frac{\|y_i\|^2}{\max(0.1\Delta_k, \sigma_k)^2} \right)^3 \cdot \ell_i(s_k)^2,$$

and replace the point that achieved the maximum score by s_k . Intuitively, this attempts to simultaneously remove points that are deemed far away while attempting to improve geometry. After such self-correction, the geometry correction is carried out in the same manner as in GC-YZ-LIN. Since the pseudocode closely mirrors Algorithm 5 and is not currently accompanied by any theoretical guarantees, we omit it and present the results as a proof of concept. We see that this algorithm somewhat outperforms both GC-YZ-LIN and NEWUOA and believe that modifications to only the self-correction step can be ultimately covered by extending our theory.

The random subspace variation with geometry correction has a straightforward implementation, in that there is little deviation from the theoretical framework described in Algorithm 4. We call this algorithm GC-sub. The only deviation from the framework is the addition of a successful and unsuccessful iteration “patience” parameter. This value dictates the required number of successful or unsuccessful iterations before a subspace is redrawn. Intuitively, if we are making significant progress on a particular subspace, it may be beneficial to keep this subspace instead of redrawing. In our theoretical framework Algorithm 4 this parameter is set to 1. While one can extend the theory to any fixed value of this parameter, it would complicate the notation of Section 5. On the other hand our computational results suggest that 1 is the best value for this parameter at least in the current setting.

Algorithm 5: Geometry-correcting algorithm with $\mathcal{Y}$ and $\mathcal{Z}$ separation (GC-YZ-LIN)

Inputs: A zeroth-order oracle $f(x) \approx \phi(x)$, $\Delta_0, x_0, \gamma \in (0, 1)$, $\eta_1 > 0, \eta_2 > 0, \Lambda > 1, \Lambda_{sc} \geq 1, \theta \in (0, \gamma)$, $\sigma_{\text{end}} \in (0, \Delta_0]$, $c_g \in (0, 1]$, model Hessian bound K , averaging weight $\beta \in (0, 1]$.

Initialization: An initial set $\mathcal{Y}_0$ such that $|\mathcal{Y}_0| = n$, an initial set $\mathcal{Z}_0$ such that $|\mathcal{Z}_0| \leq n(n+1)/2$ and the function values $f(x_0), f(x_0 + y_i), y_i \in \mathcal{Y}_0, f(x_0 + z_i), z_i \in \mathcal{Z}_0$. Set the resolution floor $\sigma_0 = \Delta_0$. A set of Lagrange polynomials $\{\ell_i(x), i = 1, \dots, n\}$ in $\mathcal{P}$ for the set $\mathcal{Y}_0$, model errors $e_C, e_F \leftarrow \text{undefined}$, $H_{\text{prev}} \leftarrow \mathbf{0}$, set active model label $\alpha \leftarrow F$.

for $k = 0, 1, 2, \dots$ **do**

- 1 *Model building:* Construct MFN model g_F, H_F . Construct MCFN model g_C, H_C . Set $H_{\text{prev}} = H_C$.
- 2 If e_C or e_F is undefined, then $\alpha \leftarrow F$ and skip *model switching* step in Line 3.
- 3 *Model switching:* If $\alpha = F$ and $e_C < 0.8e_F$ then $\alpha \leftarrow C$, otherwise if $\alpha = C$ and $e_F < 0.8e_C$ then $\alpha \leftarrow F$.
- 4 If $\alpha = F$ then $g_k, H_k \leftarrow g_F, H_F$. If $\alpha = C$ then $g_k, H_k \leftarrow g_C, H_C$.
- 5 *Ensuring bounded model Hessians:* If $\|H_k\| > K$ set g_k, H_k by solving (4.1).
- 6 Construct the set of Lagrange polynomials $\{\ell_i(x), i = 1, \dots, n\}$ in $\mathcal{P}$ for the set $\mathcal{Y}_k$ and find max value

$$(j_k^*, s_k^*) = \arg \max_{j=1, \dots, n, s \in B(0, \Delta_k)} |\ell_j(s)|.$$
- 7 *Small model gradient gate:* If $\|g_k\| < \eta_2 \Delta_k$ perform the following checks:
 If $|\ell_{j_k^*}(s_k^*)| > \Lambda$ then perform “Geometry correction of $\mathcal{Y}$ by replacing a “bad” point”.
 If $|\ell_{j_k^*}(s_k^*)| \leq \Lambda$ and $\Delta_k \leq \sigma_k$ perform the *resolution update*:
 $\sigma_{k+1} = \max\{\theta \sigma_k, \sigma_{\text{end}}\}, \Delta_{k+1} = \max\{\frac{1}{2} \Delta_k, \sigma_{k+1}\}.$
 If $|\ell_{j_k^*}(s_k^*)| \leq \Lambda$ and $\Delta_k > \sigma_k$, then $\Delta_{k+1} \leftarrow \max\{\gamma \Delta_k, \sigma_k\}.$
 Go to 1 and $k \leftarrow k + 1$.
- 8 Compute a trial step s_k as in Algorithm 1.
- 9 *Small step gate:* If $\|s_k\| < c_g \sigma_k$, do not evaluate $f(x_k + s_k)$. Perform the same checks as *small model gradient gate*.
- 10 Evaluate $f(x_k + s_k)$ and compute the ratio ρ_k as in Algorithm 1.
- 11 *Model scoring:* **for** $\mu \in \{F, C\}$ **do**

$$p_\mu \leftarrow (g_\mu)^\top s_k + \frac{1}{2} s_k^\top H_\mu s_k, \quad f_c \leftarrow f(x_k + s_k) - f(x_k)$$

$$\varepsilon_\mu \leftarrow \frac{|p_\mu - f_c|}{\max\{|f_c|, |p_\mu|\}}$$
 If e_μ is undefined, then $e_\mu = \varepsilon_\mu$, otherwise $e_\mu = \beta e_\mu + (1 - \beta) \varepsilon_\mu$.
- 12 *Successful iteration:* $\rho_k \geq \eta_1$. Perform the successful update as in Algorithm 2.
- 13 *Unsuccessful or Model Improving iteration:* $\rho_k < \eta_1$. Perform all applicable steps (i), ..., (iv) from Algorithm 2.
Model improving iteration: If $I_{\text{imp}} = 1$, set $\Delta_{k+1} = \max\{\gamma \Delta_k, \sigma_k\}.$
Unsuccessful iteration above the floor: If $I_{\text{imp}} = 0$ and $\Delta_k > \sigma_k$, set $\Delta_{k+1} = \max\{\gamma \Delta_k, \sigma_k\}.$
Unsuccessful iteration at floor: If $I_{\text{imp}} = 0$ and $\Delta_k \leq \sigma_k$, perform the *resolution update* from Line 7.

6.2 Testing and Results We now turn to the results. We state upfront that our numerical comparison yields the following observations.

1. The new adaptive Hessian fitting procedure improves performance for all solvers.
2. Methods with geometry correction, i.e. GC-YZ-LIN/V and NEWUOA, outperform DFOTR in higher dimensions.

3. Using the $\mathcal{Y}$ - $\mathcal{Z}$ separation, GC-YZ-LIN is able to largely match NEWUOA, with trade-offs between early-game and late-game performance. Transplanting NEWUOA’s self-correcting rule into the $\mathcal{Y}$ - $\mathcal{Z}$ framework, we obtain a variant that outperforms NEWUOA; no theoretical analysis has been developed for it yet.
4. A practical implementation of a randomized subspace algorithm that is competitive with full space solvers remains difficult.

Here we give the details of our testing methodology. Table 6.1 outlines the parameters used for our proposed solver and DFOTR. We will use PRIMA’s [27] implementation of NEWUOA, which has fixed trust region parameters by default, so we omit restating it in the table. All problems are initialized in the same way, where the initial interpolation set $\mathcal{Y}_0$ is set to $\{\Delta_0 u_i\}$ where $u_i, i = 1, \dots, n$ are randomly rotated coordinate vectors, and radius $\Delta_0 = 0.5$. $\mathcal{Z}_0$ is then set to $\{-\Delta_0 u_i\}$. To allow the algorithm to run until an exhausted budget, we set the termination criteria to be $\Delta_{\text{end}} = 10^{-12}$ for DFOTR and $\rho_{\text{end}}, \sigma_{\text{end}} = 10^{-12}$ for NEWUOA and GC-YZ-LIN/V respectively. All trust region subproblems are solved exactly. We compare performances using data profiles at tolerances $\tau \in \{10^{-2}, 10^{-4}, 10^{-6}\}$ and report the area under curve score of each solver in the legend. We average the curves obtained from the different randomly rotated initial sets.

We remark on the choice of three parameters in GC-YZ-LIN that also show up in the complexity analysis. Firstly, the model Hessian bound K is set to be extremely large, namely 10^{100} and essentially inactive. This is largely a byproduct of testing solver performance on problems from CUTEst with naturally extreme curvature. For example, problems such as “SSBRYND” and “SCURLY10” have true Hessian norms greater than 10^{20} while “POWERSUM” can have a true Hessian norm even exceeding 10^{100} . Thus setting a universal, fixed, and small K will handicap the model’s ability to capture the function’s true curvature. Additionally, we set η_2 to be small, namely 5×10^{-9} . While ensuring the norm of the model gradient is not too small relative to the radius is necessary for the theory, a large η_2 tethering the trust region radius to the model gradient can be detrimental to problems with small local Lipschitz constants, for example, the Rosenbrock function outside of the “banana valley”. If there is still progress to be made, iterates in this region will be forced to have a small trust region radius and may stunt progress. Moreover, in practice it appears beneficial to have a relaxed choice for Λ . According to our theory, Λ -poisedness guarantees a certain amount of function decrease on successful iterations each time. However, ensuring a small Λ in practice comes at the cost of more function evaluations for geometry correction. We observe that the best performance comes when $\Lambda = 1000$, which suggests that although the geometry may not be perfect, the algorithm is still able to progress, and geometry correction should only take place when the geometry is exceedingly bad. We also make the distinction between Λ and Λ_{sc} , where the latter is a threshold on how large a Lagrange polynomial value at a step would need to be to initiate a self-correcting step. In practice we set it to 2, while the analysis applies as long as $\Lambda_{sc} \geq 1$.

These seemingly detrimental parameter choices for η_2 and Λ in terms of the complexity bounds can be explained by the fact that our analysis addresses the worst-case. Setting $\eta_2 = \sqrt{n}$ and $\Lambda = \Theta(1 + \frac{1}{n})$ essentially forces more model improving iterations before a successful step is allowed or trust region radius is reduced, to ensure best models and progress according to the worst case κ_{eg} constant. However, in practice the error between the model and the function at the trial step can be much smaller than the worst case κ_{eg} bound suggests, so too many model improvement steps may be wasteful. It would be interesting to explore adaptive choices for these constants in future work.

Low-dimensional tests. We begin our testing on 174 low-dimensional test problems with dimension between $2 \leq n \leq 5$. Since the dimension is low, we set the maximum allowable number of points in the sample set to be $(n+1)(n+2)/2$ points. Aside from the first few iterations, the sample set of GC-YZ-LIN/V and NEWUOA will eventually become full, resulting in an interpolation system that is fully determined, so the two models coincide and the switching is inactive. In contrast, DFOTR often removes several sample points at once, so the hedge model can be active throughout the trajectory. Figure 6.1 shows the performance of the solvers on this test set. All solvers seem to perform comparably well.

Medium-/High-dimensional tests. We also benchmark on test sets in dimension 30, 100, and 200, with 95, 101, and 92 problems respectively in each set. For these tests, we will use a maximum of $2n+1$ sample points in the total interpolation set. Firstly, Figure 6.2 compares the solvers using their default fitting routine against using the hedge model. We see that adopting this model strictly improves performance across the board. In what follows, to ensure fair testing, all solvers are benchmarked using the hedge model.

Figures 6.3, 6.4, and 6.5 show the data profiles for the 30-, 100- and 200- dimensional test sets respectively.

Table 6.1: Solver settings used throughout testing.

	GC-YZ-LIN/V	GC-sub	DFOTR
Acceptance ratio η_1	0.01	0.1	0.05
Model gradient threshold η_2	5×10^{-9}	5×10^{-9}	—
“Very successful iteration” threshold	—	—	0.5
Expansion factor γ_{inc}	1.3	1.3	1.6
Contraction factor γ_{dec}	0.8	0.8	0.8
Resolution shrink θ	0.1	—	—
Poisedness threshold Λ	1000	1000	—
Self-correcting threshold Λ_{sc}	2	2	—
Hedge: EWMA decay β	0.8	—	—
Model Hessian bound K	10^{100}	—	—
Small step gate threshold c_g	0.5	—	—
Subspace dimension q	—	5	—
Interpolation set capacity	—	$2q$	—

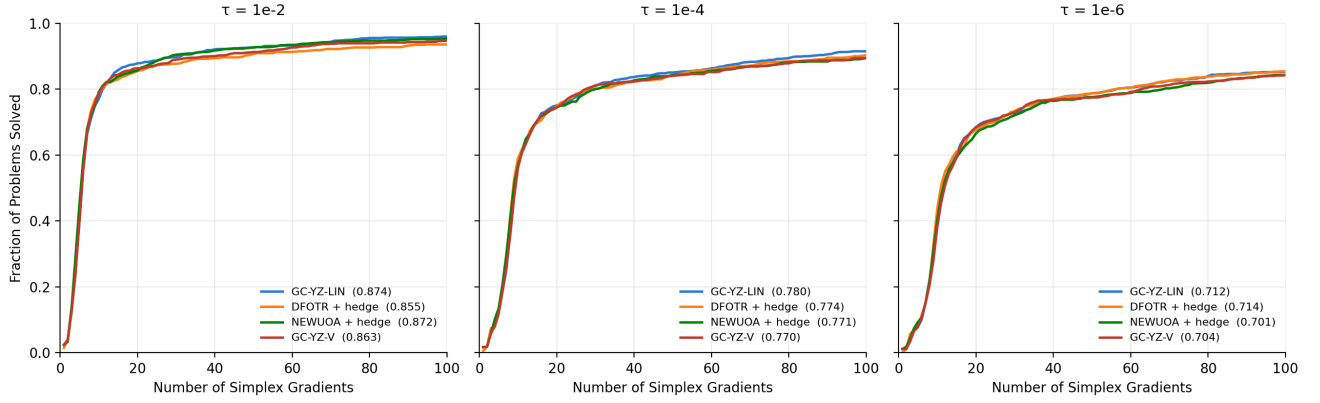


Figure 6.1: Data profiles for the low-dimensional test suite: $2 \leq n \leq 5$.

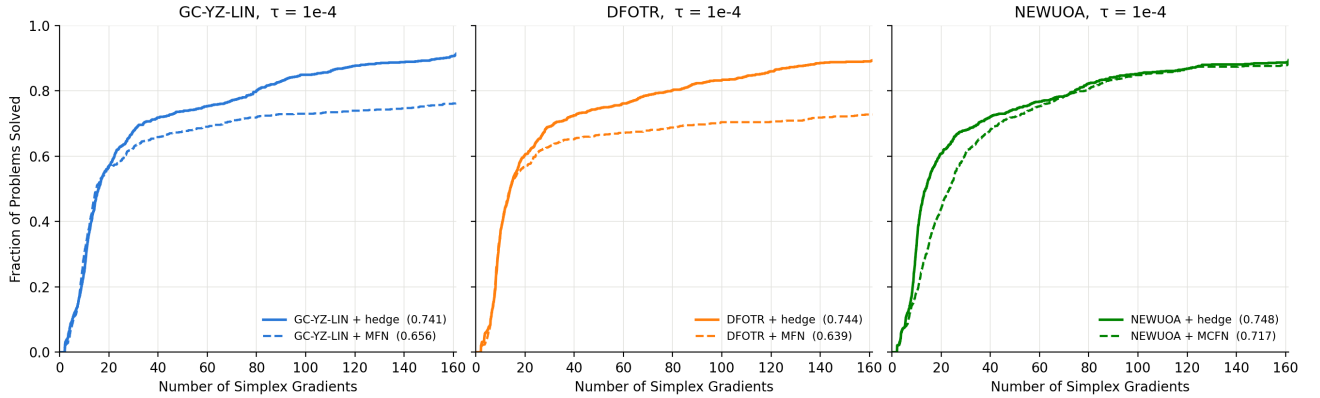


Figure 6.2: Data profiles comparing the hedge model with each solver's default fitting procedure.

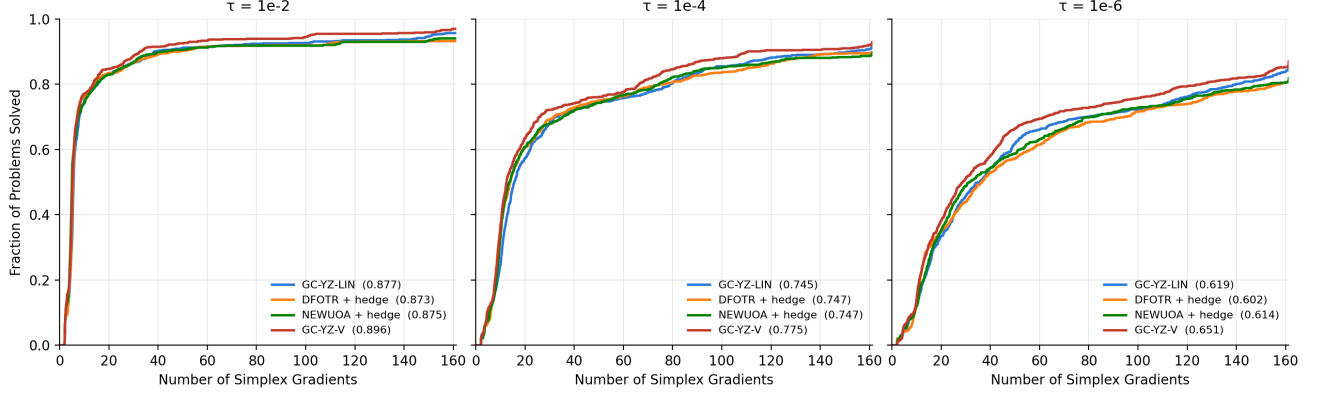


Figure 6.3: Data profiles for the test suite $n = 30$.

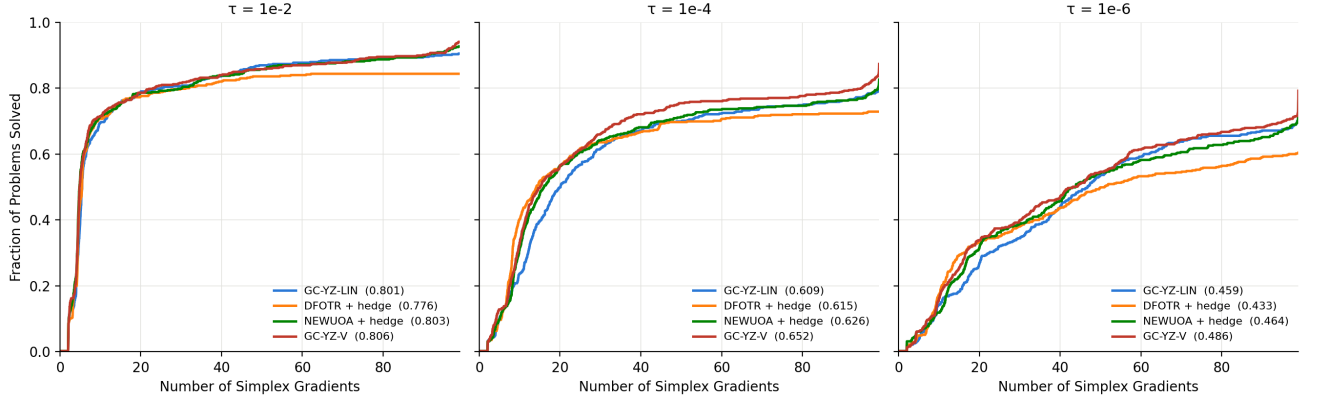


Figure 6.4: Data profiles for the test suite $n = 100$.

The first most apparent observation is the deterioration of DFOTR as dimension increases. This is largely due to the absence of a geometry management procedure. As dimension increases, the $2n + 1$ points become increasingly sparse and are more likely to be nearly degenerate. Thus, DFOTR performs increasingly aggressive shrinking to eventually force a geometry reset, which leads to radius collapse and premature termination, as seen in the significant flattening of the DFOTR data profile curves.

Another apparent observation is the almost strict improvement of GC-YZ-V over NEWUOA and GC-YZ-LIN. GC-YZ-V is a combination of several algorithmic features from both NEWUOA and GC-YZ-LIN. This result suggests that the use of NEWUOA-style self-correction may be more efficient than first ensuring all points are close, then checking the self-correction property. While the latter is more principled and easier to analyze theoretically, it may be slower in practice. This also suggests it may be worthwhile to maintain a separation of sets $\mathcal{Y}$ - $\mathcal{Z}$, even when quadratic Lagrange polynomials are maintained for the entire sample set $\mathcal{Y} \cup \mathcal{Z}$. Again, we note that this variant is not covered by current theory and is a line of future work.

Finally, we observe that GC-YZ-LIN is closely competitive with NEWUOA across dimensions and across tolerances. At $\tau = 10^{-2}$, the curves are nearly identical. At $\tau = 10^{-4}, 10^{-6}$, a pattern emerges where NEWUOA is quicker to solve more problems early in the run, however GC-YZ-LIN eventually catches up and overtakes NEWUOA later in the run. In general, the performance of the two methods is comparable, which is surprising given that GC-YZ-LIN only manages linear geometry for a subset of the sample set compared to NEWUOA's full quadratic geometry.

Results in subspaces. Figure 6.6 compares the performance of the random subspace method against GC-YZ-LIN. The subspace method sets $q = 5$, patience parameters equal to 1, and draws a fresh coordinate stencil on every subspace redraw. A maximum of $2q + 1$ points is used in the sample. We remark that varying choices of

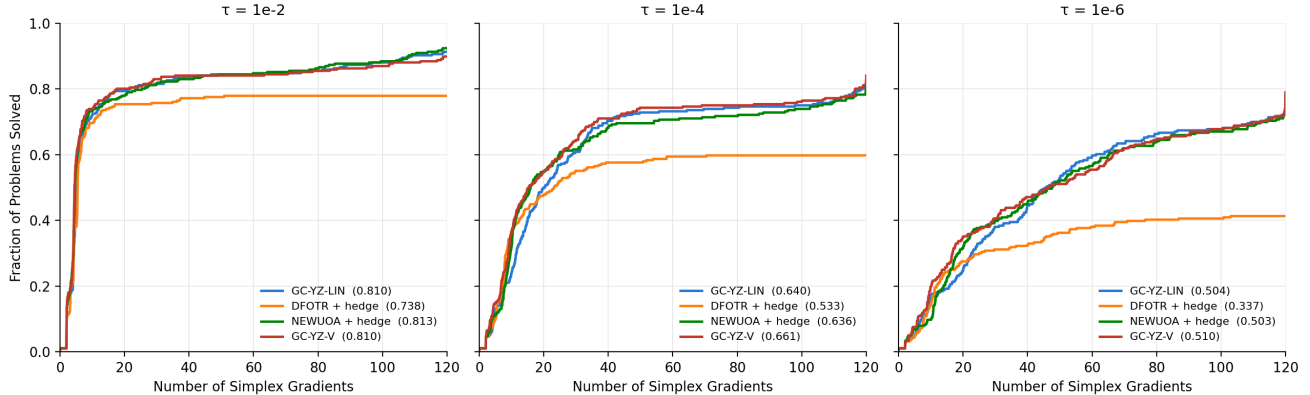


Figure 6.5: Data profiles for the test suite $n = 200$.

patience parameter, subspace dimension, re-draw strategy, and maximum number of allowed sample points were tested, and we opt to present the best performing combination found in dimension 30 and 100. It is evident that the subspace method does not perform as well as a full space solver and its performance deteriorates as tolerances get smaller. While theory suggests that as dimension increases, the competitiveness of subspace methods should as well, this is not something that is immediately evident. A plausible explanation for the discrepancy between theory and practice is as follows. The random subspace method needs at least $\mathcal{O}(q)$ points for each subspace redraw and the bound on the number of subspace redraws is not loose. Thus we arrive at the worst case complexity of order q without fail. However, in the full space solvers, we rarely need more than a few consecutive geometry correcting steps to achieve good geometry, hence in practice we are typically far from actually attaining the worst-case behavior described in the complexity bound. Another explanation is that subspace methods ultimately lose the advantage of quadratic approximation in the whole space, so may only be competitive where full space quadratic approximation is prohibitive or useless. A future implementation which initiates any sample set with as few as two sample points may improve performance of subspace methods. In addition reusing sample points from other subspaces can be beneficial. In conclusion, our contribution in terms of subspace TR method in this paper is mainly theoretical while practical approaches require further investigation. We would like to point the reader to the extensive empirical study carried out in [8].

Acknowledgments This work was partially supported by ONR award N00014-22-1-215 and the Gary C. Butler Family Foundation.

References

- [1] Charles Audet and Warren L. Hare. *Derivative-Free and Blackbox Optimization*. Springer Series in Operations Research and Financial Engineering. Springer International Publishing, Cham, 2017.
- [2] Afonso S Bandeira, Katya Scheinberg, and Luís N Vicente. Computation of sparse low degree interpolating polynomials and their application to derivative-free optimization. *Mathematical Programming*, 134(1):223–257, 2012.
- [3] Afonso S Bandeira, Katya Scheinberg, and Luís N Vicente. Convergence of trust-region methods based on probabilistic models. *SIAM Journal on Optimization*, 24(3):1238–1264, 2014.
- [4] Albert S Berahas, Liyuan Cao, Krzysztof Choromanski, and Katya Scheinberg. A theoretical and empirical comparison of gradient approximations in derivative-free optimization. *Foundations of Computational Mathematics*, pages 1–54, 2021.
- [5] Albert S Berahas, Liyuan Cao, and Katya Scheinberg. Global convergence rate analysis of a generic line search algorithm with noise. *SIAM Journal on Optimization*, 31(2):1489–1518, 2021.
- [6] Jose Blanchet, Coralia Cartis, Matt Menickelly, and Katya Scheinberg. Convergence rate analysis of a stochastic trust-region method via supermartingales. *INFORMS Journal on Optimization*, 1(2):92–119, 2019.

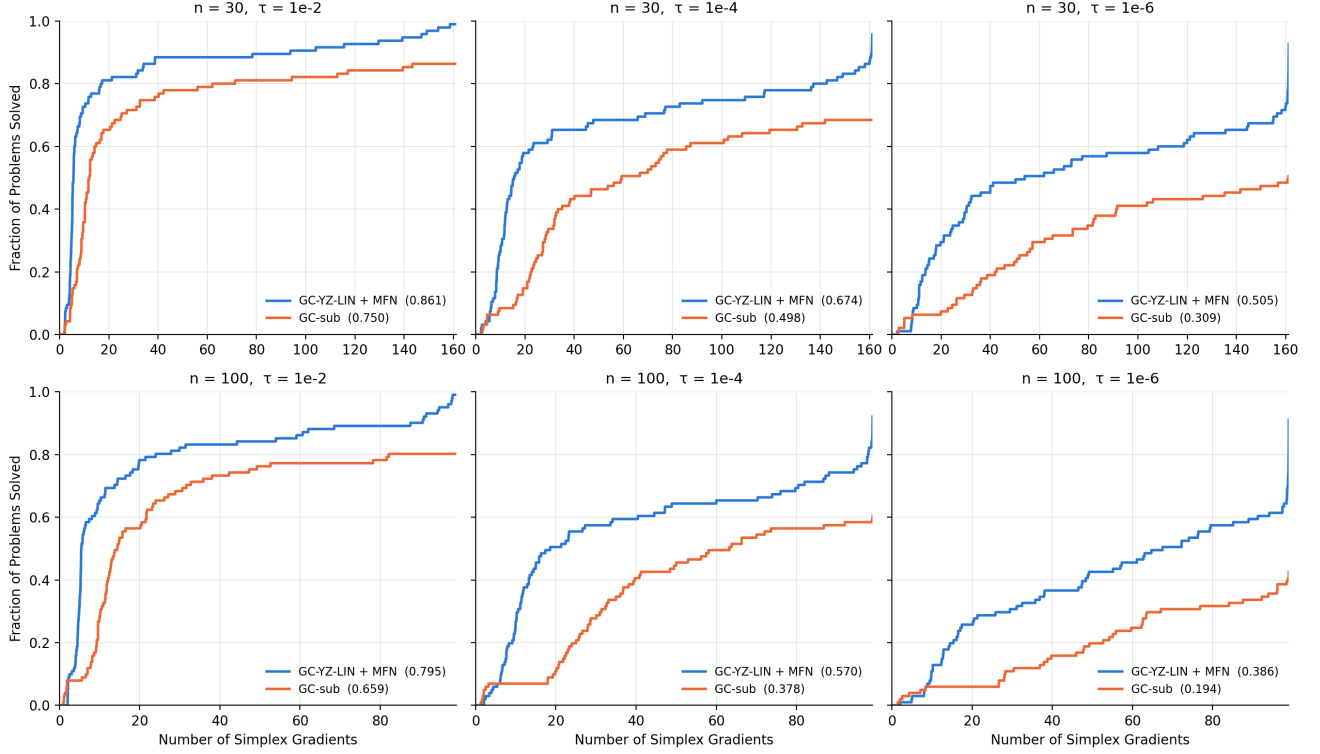


Figure 6.6: Data profiles for the random subspace implementation compared against GC-YZ-LIN for $n = 30, 100$.

- [7] Liyuan Cao, Albert S. Berahas, and Katya Scheinberg. First- and second-order high probability complexity bounds for trust-region methods with noisy oracles. *Mathematical Programming*, 207(1-2):573–624, 2023. Also available as a preprint at arXiv, May 2022.
- [8] Coralia Cartis and Lindon Roberts. Scalable subspace methods for derivative-free nonlinear least-squares optimization. *Mathematical Programming*, 199(1):461–524, May 2023.
- [9] Coralia Cartis and Lindon Roberts. A note on the complexity of random subspace model-based methods for derivative-free optimization, 2026.
- [10] Coralia Cartis and Katya Scheinberg. Global convergence rate analysis of unconstrained optimization methods based on probabilistic models. *Mathematical Programming*, 169(2):337–375, 2018.
- [11] Abraar Chaudhry and Katya Scheinberg. On complexity of model-based derivative-free methods. In *Proceedings of the International Congress of Mathematicians (ICM) 2026*, volume 7, pages 208–228, Philadelphia, PA, USA, 2026. SIAM.
- [12] Andrew R Conn, Nicholas IM Gould, and Philippe L Toint. *Trust region methods*. SIAM, 2000.
- [13] Andrew R Conn, Katya Scheinberg, and Luís N Vicente. *Introduction to derivative-free optimization*. SIAM, 2009.
- [14] Kwassi Joseph Dzahini and Stefan M. Wild. Stochastic trust-region algorithm in random subspaces with convergence and expected complexity analyses. *SIAM Journal on Optimization*, 34(3):2743–2774, 2024.
- [15] R. Garmanjani, D. Júdice, and L. N. Vicente. Trust-region methods without using derivatives: Worst case complexity and the non-smooth case. *SIAM Journal on Optimization*, 26(4):1987–2011, 2016.

- [16] Serge Gratton, Clément W Royer, Luís N Vicente, and Zaikun Zhang. Complexity and global rates of trust-region methods based on probabilistic models. *IMA Journal of Numerical Analysis*, 38(3):1579–1597, 2018.
- [17] Serge Gratton and Philippe L. Toint. S2mpj and cutest optimization problems for matlab, python and julia, 2024.
- [18] Billy Jin, Katya Scheinberg, and Miaolan Xie. High probability complexity bounds for line search based on stochastic oracles. *Advances in Neural Information Processing Systems*, 34:9193–9203, 2021.
- [19] Jeffrey Larson, Matt Menickelly, and Stefan M Wild. Derivative-free optimization methods. *Acta Numerica*, 28:287–404, 2019.
- [20] Jorge J Moré and Stefan M Wild. Benchmarking derivative-free optimization algorithms. *SIAM Journal on Optimization*, 20(1):172–191, 2009.
- [21] Michael J D Powell. The NEWUOA software for unconstrained optimization without derivatives. Technical Report DAMTP 2004/NA08, Department of Applied Mathematics and Theoretical Physics, University of Cambridge, 2004.
- [22] Michael JD Powell. UOBYQA: unconstrained optimization by quadratic approximation. *Mathematical Programming*, 92(3):555–582, 2002.
- [23] MJD Powell. On the lagrange functions of quadratic models that are defined by interpolation. *Optimization Methods and Software*, 16(1-4):289–309, 2001.
- [24] T. M. Ragonneau and Z. Zhang. PDFO: a cross-platform package for Powell’s derivative-free optimization solvers. *Mathematical Programming Computation*, 16(4):535–559, 2024.
- [25] Luis Miguel Rios and Nikolaos V. Sahinidis. Derivative-free optimization: a review of algorithms and comparison of software implementations. *Journal of Global Optimization*, 56(3):1247–1293, 2013.
- [26] K. Scheinberg and Ph. L. Toint. Self-correcting geometry in model-based algorithms for derivative-free unconstrained optimization. *SIAM Journal on Optimization*, 20(6):3512–3532, 2010.
- [27] Z. Zhang. PRIMA: Reference Implementation for Powell’s Methods with Modernization and Amelioration. available at <http://www.libprima.net>, DOI: 10.5281/zenodo.8052654, 2023.